AI Agents and Applications

With LangChain, LangGraph and MCP

Roberto Infante

MANNING

AI智能体与应用

使用 LangChain、LangGraph 与 MCP

[加拿大]罗贝托·因凡特(Roberto Infante)
(著)

益达 (译)

如需评论请前往 livebook。

曼宁
谢尔特岛

欲了解关于本书的更多信息以及其他曼宁出版社出版物的信息，请访问
manning.com。

版权声明

如需在线查阅与订购本书及其他曼宁出版社出版物，请访问www.manning.com。批量订购本书可享受出版社提供的折扣优惠。欲了解更多详情，请联系

特价销售部

曼宁出版社

鲍德温路20号

邮政信箱 761

纽约谢尔特岛 11964

电子邮箱：orders@manning.com

©2026 曼宁出版社出品。版权所有。

未经出版商事先书面许可，本出版物的任何部分均不得以任何形式或通过任何方式（电子、机械、影印或其他方式）进行复制、存储于检索系统或传播。

制造商和销售商为区分其产品所使用的许多标识均被声明为商标。当这些标识出现在本书中，且曼宁出版社知晓商标声明时，这些标识会以首字母大写或全大写的形式印刷。

认识到保存文字内容的重要性，曼宁出版社的政策是确保我们出版的书籍均采用无酸纸印刷，并为此竭尽全力。同时，我们也认识到保护地球资源的责任，因此曼宁图书所采用纸张至少含有15%的再生纤维，且在加工过程中不使用氯元素。

作者与出版商已竭尽全力确保本书所载信息在付印时的准确性。对于任何第三方因本书信息错误或遗漏所导致的任何损失、损害或中断的责任，无论此类错误或遗漏是由疏忽、意外或其他原因所致，亦或源于对本书信息的任何使用，作者与出版商均不承担任何责任，特此声明。

曼宁出版社
鲍德温路 20号
邮政信箱 761
纽约谢尔特岛，邮编 11964

开发编辑：达斯汀·阿奇伯尔德(Dustin Archibald)
技术编辑：基尔蒂瓦桑·桑塔纳克里希南(Keerthivasan Santhanakrishnan)
和 安托万·马利克·巴茨(Antowan Malik Batts)
审阅编辑：基肖尔·里特(Kishor Rit)
制作编辑：凯西·罗斯兰(Kathy Rossland)
文字编辑：朱莉·麦克纳米(Julie McNamee)
校对员：梅洛迪·多拉布(Melody Dolab)
技术校对员：安德鲁·R·弗里德(Andrew R. Freed)
排版员与封面设计师：玛丽亚·图多尔(Marija Tudor)

ISBN 9781633436541

印刷于美国

献词

谨以此书献给我的母亲和父亲

目录

前言

致谢

关于本书

作者简介

关于封面插图

第一部分 大语言模型入门

1 AI智能体与应用导论

1.1 构建基于大语言模型的应用与智能体

1.1.1 基于大语言模型的应用：摘要生成与问答引擎

1.1.2 基于大语言模型的聊天机器人

1.1.3 AI智能体

1.2 LangChain 简介

1.2.1 LangChain 架构

1.2.2 LangChain的核心对象模型

1.3 大语言模型的典型应用场景

1.4 如何根据您的需求调整大语言模型

1.4.1 提示词工程

1.4.2 检索增强生成(RAG)

1.4.3 微调

1.5 选择哪种大语言模型

1.6 你从本书中能学到什么

2 以编程方式执行提示词

2.1 以编程方式运行提示词

2.1.1 本章环境设置

2.1.2 极简提示词执行

2.2 使用LangChain运行提示词

2.3 提示词模板

2.3.1 使用Python函数实现提示词模板

2.3.2 使用 LangChain 的 PromptTemplate

2.4 提示词类型

2.4.1 文本分类

2.4.2 情感分析

2.4.3 文本摘要

2.4.4 文本创作

2.4.5 问答

2.4.6 推理

2.5 详细推理

2.5.1 单样本学习

2.5.2 双样本学习

2.5.3 提供步骤

2.5.4 少样本学习

2.5.5 使用LangChain实现少样本学习

2.5.6 思维链

2.6 提示词结构

第二部分 摘要生成

3 使用LangChain进行文本摘要

3.1 为超出上下文窗口长度的文档生成摘要

3.1.1 将文本切分为文档对象

3.1.2 分割

3.1.3 映射

3.1.4 合并

3.1.5 MapReduce组合链

3.1.6 MapReduce执行

3.2 跨文档摘要

3.2.1 创建文档对象列表

3.2.2 维基百科内容

3.2.3 基于文件的内容

3.2.4 创建文档列表

3.2.5 逐步完善最终摘要

3.3 摘要流程图

4 构建研究摘要引擎

4.1 研究摘要引擎概述

4.2 项目设置

4.3 实现核心功能

4.3.1 实现网络搜索

4.3.2 实现网络爬虫

4.3.3 实例化LLM客户端

4.3.4 JSON到Python对象转换器

4.4 通过查询重写增强架构

4.5 提示词工程

4.5.1 构建web搜索提示词

4.5.2 构建摘要提示词

4.5.3 研究报告提示词

4.6 初步实现

4.6.1 导入函数与提示词模板

4.6.2 设置常量与输入变量

4.6.3 实例化LLM客户端

4.6.4 生成web搜索并收集结果

4.6.5 抓取网页结果

4.6.6 网络结果汇总

4.6.7 生成研究报告

4.7 在LCEL中重新实现研究摘要引擎

4.7.1 助手指令链

4.7.2 web搜索链

4.7.3 搜索与摘要链

4.7.4 网络研究链

5 使用LangGraph构建智能体工作流

5.1 理解智能体工作流与智能体

5.1.1 工作流

5.1.2 智能体

5.1.3 何时使用基于代理的架构

5.1.4 智能体开发框架

5.2 LangGraph 基础

5.3 从LangChain链迁移到LangGraph

5.4 LangGraph核心组件

5.4.1 状态图结构

5.4.2 状态管理与类型系统

5.4.3 节点函数与边定义

5.4.4 入口点与结束条件

5.5 将web研究助手转变为AI智能体

5.5.1 原始LangChain实现概述

5.5.2 识别待转换的组件

5.5.3 逐步转换过程

5.5.4 代码对比与实现的收益

第三部分 问答聊天机器人

6 使用ChromaDB的RAG基础

6.1 语义搜索

6.1.1 基于单一文档的基础问答聊天机器人

6.1.2 基于知识库的复杂问答聊天机器人

6.1.3 RAG设计模式

6.2 向量存储

6.2.1 什么是向量存储?

6.2.2 向量数据库是如何工作的?

6.2.3 向量库与向量数据库

6.2.4 最受欢迎的向量数据库

6.2.5 使用 Chroma 存储文本并执行语义搜索

6.3 从零开始实现RAG

6.3.1 从向量数据库中检索内容

6.3.2 调用大语言模型

6.3.3 构建聊天机器人

6.3.4 RAG术语回顾

7 使用LangChain和LangSmith构建问答聊天机器人

7.1 用于问答聊天机器人的LangChain对象模型

7.1.1 内容摄取（索引）阶段

7.1.2 问答（检索与生成）阶段

7.2 向量存储内容摄取

7.2.1 文档的分割与存储

7.2.2 消除重复

7.2.3 从文件夹批量摄取文档

7.3 跨存储文档的问答

7.3.1 直接查询向量存储

7.3.2 通过LangChain链提问

7.3.3 完成RAG链的设置

7.3.4 后续问题

7.4 聊天机器人的消息历史记忆

7.4.1 修改提示

7.4.2 更新聊天消息历史记录

7.4.3 将聊天历史记录输入RAG链

7.4.4 将所有内容整合起来

7.5 使用 LangSmith 追踪执行过程

第四部分 高级检索增强生成

8 高级索引

8.1 提升RAG准确性

8.1.1 内容摄取阶段

8.1.2 问答阶段

8.2 高级文档索引

8.3 分割策略

8.3.1 分割策略

8.3.2 需考虑的因素

8.3.3 选择正确的策略

8.3.4 按HTML标题分割

8.4 嵌入策略

8.4.1 使用ParentDocumentRetriever嵌入子文档块

8.4.2 使用 MultiVectorRetriever 嵌入子块

8.4.3 嵌入文档摘要

8.4.4 嵌入假设性问题

8.5 粒度块扩展

8.6 半结构化内容

8.7 多模态检索增强生成

9 问题转换

9.1 重写-检索-读取

9.1.1 使用原始用户问题检索内容

9.1.2 设置查询重写器链

9.1.3 使用重写后的查询检索内容

9.1.4 将所有组件整合成一个单一的RAG链

9.2 生成多个查询

9.2.1 为生成多个查询设置链

9.2.2 设置自定义多查询检索器

9.2.3 使用标准MultiQueryRetriever实例

9.3 回溯式提问

9.3.1 设置链以生成回溯问题

9.3.2 将回溯式问题生成整合到RAG链中

9.4 假设性文档嵌入（HyDE）

9.4.1 为用户问题生成假设性文档

9.4.2 将HyDE链集成到RAG链中

9.5 单步与多步分解

10 查询生成、路由与检索后处理

10.1 内容数据库查询生成

10.2 自查询（元数据查询增强）

10.2.1 数据摄取：元数据增强

10.2.2 关于元数据增强型集合的问答

10.3 生成结构化SQL查询

10.3.1 安装 SQLite

10.3.2 设置并连接到数据库

10.3.3 从自然语言生成SQL查询

10.3.4 执行SQL查询

10.4 生成语义SQL查询

10.4.1 标准SQL查询

10.4.2 语义化SQL查询

10.4.3 创建嵌入向量

10.4.4 执行语义SQL搜索

10.4.5 自动化语义SQL搜索

10.4.6 语义SQL搜索的优势

10.5 为图数据库生成查询

10.6 链式路由

10.6.1 设置数据检索器

10.6.2 设置查询路由器

10.6.3 将链式路由器集成到完整的RAG链中

10.7 检索后处理

10.7.1 相似性后处理器

10.7.2 关键字后处理器

10.7.3 时间加权

10.7.4 RAG融合（互逆排序融合）

第五部分 AI智能体

11 使用LangGraph构建基于工具的智能体

11.1 从简单开始：构建一个单工具旅行信息助手

11.1.1 项目设置

11.1.2 加载环境变量

11.1.3 准备旅行信息向量存储

11.2 启用代理调用工具

11.2.1 从函数调用到工具调用

11.2.2 工具调用如何与大语言模型协同工作

11.2.3 向LLM注册工具

11.2.4 代理状态：追踪对话

11.2.5 执行工具调用

11.2.6 LLM节点：协调推理与行动

11.3 组装代理图

11.4 理解智能体图结构

11.5 运行智能体聊天机器人：读取-求值-打印循环

11.6 执行请求

11.6.1 逐步调试

11.7 扩展你的智能体：添加天气预报工具

11.7.1 实现一个模拟天气服务

11.7.2 创建天气预报工具

11.7.3 更新代理以支持多工具

11.8 执行多功能代理

11.8.1 运行多工具代理（初始行为）

11.8.2 通过系统引导提升LLM工具使用能力

11.9 使用预构建组件进行快速开发

11.9.1 重构以使用LangGraph ReAct智能体

11.9.2 运行预构建代理

11.9.3 使用LangSmith进行观察与调试

11.9.4 启用LangSmith追踪

12 多智能体系统

12.1 构建住宿预订代理

12.1.1 酒店预订工具

12.1.2 B&B 预订工具

12.1.3 ReAct 住宿预订代理

12.2 构建基于路由的旅行助手

12.2.1 设计路由代理

12.2.2 路由逻辑

12.2.3 构建多智能体图

12.2.4 尝试使用路由代理

12.3 使用监督者组件处理多代理请求

12.3.1 监督者模式：代理的代理

12.3.2 从“单向”到“往返”交互

12.3.3 试用监督代理

13 构建与使用MCP服务器

13.1 MCP服务器简介

13.1.1 问题：大规模上下文整合

13.1.2 解决方案：模型上下文协议

13.1.3 MCP生态系统

13.2 如何构建MCP服务器

13.2.1 MCP服务器开发必备资源

13.2.2 官方语言特定的MCP SDK

13.2.3 在LLM应用与智能体中集成MCP服务器

13.3 构建一个天气MCP服务器

13.3.1 实现MCP服务器

13.3.2 使用 MCP Inspector 测试 MCP 服务器

13.3.3 从测试MCP主机消费MCP服务器

13.4 将天气MCP工具集成到智能体中

13.4.1 为旅行代理准备实时天气数据

13.4.2 集成AccuWeather MCP工具

13.4.3 更新智能体对话循环

13.4.4 本地与远程工具的结合使用

13.4.5 测试与验证

13.4.6 使用代理进行复杂查询

14 生产化AI智能体：记忆、防护栏及其他

14.1 内存

14.1.1 内存类型

14.1.2 为何需要短期记忆

14.1.3 LangGraph中的检查点

14.1.4 为我们的旅行助手添加短期记忆

14.1.5 执行启用检查点的助手

14.1.6 将状态回滚至历史检查点

14.2 防护栏

14.2.1 实施防护栏以拒绝非旅行相关问题

14.2.2 在代理层面实施更严格的护栏机制

14.3 本章之外

14.3.1 长期用户与应用内存

14.3.2 人在回路

14.3.3 模型后置防护栏

14.3.4 AI智能体与应用的评估

14.3.5 在LangGraph平台与开放智能体平台上的部署

附录A 试用LangChain

A.1 在Jupyter Notebook环境中试用LangChain

A.1.1 句子补全示例

A.1.2 提示词工程示例

A.1.3 使用LCEL创建链并执行

附录B 设置Jupyter Notebook环境

B.1 安装Python解释器或Python发行版

B.2 使用venv创建虚拟环境并升级pip

B.3 设置Jupyter Notebook

附录 C 选择大语言模型

C.1 主流大语言模型

C.1.1 OpenAI GPT 系列

C.1.2 Gemini

C.1.3 Gemma

C.1.4 Claude

C.1.5 Cohere

C.1.6 Llama

C.1.7 Falcon

C.1.8 Mistral

C.1.9 千问(Qwen)

C.1.10 Grok

C.1.11 Phi

C.1.12 DeepSeek

C.2 如何选择模型

C.2.1 模型的定位

C.2.2 专有软件与开源软件

C.2.3 模型大小（参数数量）

C.2.4 上下文窗口大小

C.2.5 多语言支持

C.2.6 精度与速度的权衡

C.2.7 成本与硬件要求

C.2.8 任务适用性（标准基准测试）

C.2.9 安全性与偏见

C.2.10 一个实际示例

C.3 注意事项

附录 D 在Windows上安装SQLite

D.1 安装 SQLite

附录 E 开源大语言模型

E.1 开源大语言模型的优势

E.1.1 透明度

E.1.2 隐私

E.1.3 社区驱动

E.1.4 成本节约

E.2 流行的开源大语言模型

E.3 关于本地运行开源大语言模型的考量

E.3.1 消费级硬件的局限性

E.3.2 量化

E.3.3 OpenAI REST API 兼容性

E.4 本地推理引擎

E.4.1 llama.cpp

E.4.2 Ollama

E.4.3 vLLM

E.4.4 llamafile

E.4.5 LM Studio

E.4.6 本地人工智能

E.4.7 GPT4All

E.4.8 本地推理引擎比较

E.4.9 选择本地推理引擎

E.5 通过Hugging Face Transformers库进行推理

E.5.1 Hugging Face Transformers 库

E.5.2 LangChain的Hugging Face流水线

E.6 构建本地摘要引擎

E.6.1 选择推理引擎

E.6.2 启动 OpenAI 兼容服务器

E.6.3 修改原始解决方案

E.6.4 通过本地LLM运行摘要生成引擎

E.6.5 OpenAI与本地LLM的对比

索引

前言

2022年末，形势发生了变化。大语言模型（LLMs）不再像是实验性的演示，开始变得真正有用起来。一个总结段落的快速尝试，演变成了能够回答问题的聊天机器人；一个小脚本，变成了其他团队都想尝试的服务。不久之后，LLMs不再仅仅是新奇事物—它们已成为软件开发工具包中不可或缺的一部分。

大语言模型之所以如此令人振奋，原因在于：它能让软件能够"说人话"。它们可以修订合同、将日志转化为有意义的答案、编写代码并规划下一步行动—然后调用正确的工具和数据来实际完成任务。结合检索和工具使用，应用程序不再感觉像一台僵化的机器，而开始更像是一个协作伙伴。其潜力是巨大的，但将这种潜力转化为生产系统并非易事。它仍然需要细致的工作来整合数据流、设计高效的提示词、通过检索来为答案提供依据、编排多步骤工作流，并在部署后持续监控系统行为。

我进入这个领域的历程与许多开发者类似。起初，我在Jupyter Notebook中进行实验，探索各种API，了解模型在哪些方面表现出色，又在哪些方面存在不足。这些早期的探索逐渐演变成工作中基于智能体的小型业余项目，为少数寻求效率提升的早期采用者而构建。随着技术的进步—通过OpenAI API的改进、LangChain功能的不断增强、LangGraph的编排能力，以及诸如高级检索增强生成（RAG）和ReAct等新兴技术的出现—这些原型变得更加复杂精妙。与此同时，我开始撰写这本书。不止一次，我刚完成一章的内容，在几周之后便显得过时了。这既令人兴奋，有时也让人感到疲惫。

那段经历深刻影响了本书的写作方式。与其追逐每一个新参数或昙花一现的"最佳实践"，本书的侧重点在于那些已被证明稳定可靠、并构成可靠LLM应用基础的概念、架构和设计模式。我们将构建具体的系统—引擎、聊天机器人和智能体—但目标是提供可复用的基础框架：

如何结构化检索、设计提示词、组合链路、评估行为，以及清晰而自信地编排多步骤工作流。

框架在这一过程中扮演着至关重要的角色。LangChain 标准化了核心组件—比如加载器、分割器、嵌入、检索器、向量存储和提示词—使开发者无需为每个项目重复造轮子。LangGraph 在此基础上将工作流结构化为图，并协调智能体循环，而LangSmith 则提供可观测性，支持调试与评估。三者协同，使开发者能够专注于应用本身，而非底层基础设施。

为何选择这本书，又为何是现在？LLM系统构建的基础框架终于趋于稳定，足以系统性地传授；而现实需求也前所未有地迫切。开发团队急需实用的指导，能够弥补构想与实现之间的鸿沟，同时避免被单一厂商或昙花一现的技术所绑定。如果本书取得成功，你将收获一套清晰的心智模型，用于构建大语言模型驱动的系统；一组经过验证、可信赖的设计模式，可复用于各类项目，以及一种坚定的信心-即使技术生态持续快速演进，你仍能从容前行、持续构建。

致谢

撰写这样一本书绝非一人之功，我深感庆幸能在这段旅程中有如此多才华横溢且慷慨无私的人相伴左右。首先，我要感谢胡安-毛罗·博扎诺、罗穆卢斯·科尔内亚努、卡罗琳·高和伊万·林，在最初的章节还仅仅停留在提纲阶段时，他们就提供了深思熟虑的反馈。他们的评论与交流帮助塑造了本书早期的方向。我也要非常感谢德乌斯托大学的大卫·布扬和迭戈·洛佩斯-德-伊皮尼亚，尤其是他们在诸多构想尚未成型的早期草稿阶段，激励我以更高的精确性与严谨性来打磨这项工作。

特别感谢曼宁出版社的迈克尔·斯蒂文斯，他从最新第一次交谈起就对这个项目抱有坚定信念。当时，大语言模型才刚刚开始吸引全球关注，前方的道路尚不明朗。他的支持，加上随着技术发展而提供的及时指导和鼓励，帮助本书始终保持了相关性与聚焦性。

我尤其要特别感谢我的编辑达斯汀·阿奇伯尔德，他稳健的指导和深思熟虑的建议贯穿了我写作的每一个阶段；也要感谢安德鲁·弗里德，他敏锐的技术见解使诸多解释更加清晰和准确。同样非常感谢技术编辑安托万·巴茨。安托万是一位金融系统分析师和应用经济学家，在金融、数据分析和全球供应链方面拥有专业知识。他教授经济学，并致力于将分析洞察与现实商业挑战相连接。

致审阅者们—阿卜杜拉·阿尔·伊姆兰、阿布舍克·古哈、安德烈斯·马里斯卡尔、安托万·马利克·巴茨、阿图尔·古贾、阿扬·贾顿、巴拉吉·文卡塔斯瓦兰、博尔科·久尔科维奇、拜伦·加尔布雷思、卡内尔·格林菲尔德、大卫·卡斯韦尔、大卫·雅各布斯、德里克·摩根、费利佩·P·库蒂尼奥、加内什·斯瓦米纳坦、吉瓦吉斯·乔治、纪尧姆·阿莱昂、希瑟·S·沃德、霍布森·莱恩、小詹姆斯·布莱克、约翰·鲍威尔、卡尔-古斯塔夫·卡拉斯马、克里斯蒂安·博罗斯、洛拉·维森特、卢西恩-保罗·托尔热、卢克·库普卡、马纳斯·塔鲁克达尔、马可·马森齐奥、米图·马尔霍特拉、纳雷什·武鲁孔达、尼古拉斯·莫德里克、彼得·V·亨斯托克、皮

奥特·亚斯特泽布斯基、普拉桑特·乔西尤拉、拉马尼·纳塔拉詹、劳尔·西奥泰斯库、罗曼·费迪茨基、希文德拉·斯里瓦斯塔瓦、斯里哈尔沙·安纳马内尼、斯特凡诺·洛蒂尼、苏迈拉·阿夫扎尔、西奥·德斯普迪斯、维诺特·纳格什瓦兰、沃尔特·亚历山大·马塔·洛佩斯，以及许多其他人一感谢你们的仔细阅读和宝贵反馈。你们的建议促使我思考得更深入、表达得更清晰，并不断完善支撑本书的核心理念。正是因为你们的贡献，这本书才更加出色。

我还要感谢那些曾与我共同参与真实世界大语言模型项目的众多同事与合作者。深入的探讨、热烈的辩论，以及无数个“如果我们试试这个会怎样？”的瞬间，共同塑造了我对如何有效设计与部署AI智能体的思考。这些宝贵的经验，已融入本书后续的字里行间。

最后，我要将最深切的谢意献给我的家人。致我的妻子埃斯特雷拉，以及我的女儿比安卡和克利奥：感谢你们的耐心、支持以及永不褪色的爱。感谢你们在我连续数日埋头写作或修改时给予的理解。没有你们，这本书根本就不可能存在。

关于本书

*AI智能体与应用*是一本注重实践、以项目为核心的指南。我们首先介绍使大语言模型（LLM）应用在现实场景中有效运行的基础技能：设计可复用的提示词，以及利用检索增强生成（RAG）技术将模型输出基于您自己的数据。在此基础上，我们将深入探讨智能体工作流和多智能体系统，这些系统能够在单一提示词不足时，使用工具、做出决策并进行协作。

你将学会如何自信地追踪和调试你的系统，将脆弱的原型转变为可维护、可部署的应用程序。我们将使用LangChain来构建可组合的组件，使用LangGraph来创建清晰且可测试的控制流（特别是对于基于智能体的解决方案），并使用模型上下文协议（MCP）来像集成本地能力一样轻松地集成外部能力。在整本书中，我们将探讨这些系统的优势所在、它们的不足之处，以及如何解决诸如检索不一致、查询不精确、工具调用不可靠和行为漂移等常见问题。最终目标很明确：帮助你构建用户能够信任的AI解决方案。

本书适合哪些读者

本书面向希望使用LangChain、LangGraph和MCP构建生产级AI智能体和应用程序的开发者和技术型读者。如果你熟悉基本的Python和标准开发者工作流程—例如在终端中运行命令和管理虚拟环境—你会感到得心应手。熟悉在Jupyter Notebook中启动项目或在Visual Studio Code（VS Code）中开发Python应用程序将有助于你更快地进步。虽然不需要具备LLM应用开发的经验，但如果你曾尝试过ChatGPT、Claude或Gemini等工具，将会大有裨益。

您可能是一位希望超越简单大语言模型API调用的软件工程师，一位寻求将RAG原型转化为可靠应用或智能体的数据科学家或机器学习工程师，亦或是一位正在评估可行性及部署策略的技术主管或产品经

理。学生、独立学习者、研究人员和爱好者同样能在此找到一条从基础概念到完整功能系统的清晰实用路径。

在本书各章节中，您将从基础概念逐步进阶至构建功能完备的应用程序：设计高效的提示词；使用LangChain构建、测试和强化RAG管道；通过LangGraph实现智能体工作流与多智能体协同；并借助MCP集成外部工具以创建安全、可互操作的能力。在此过程中，您将逐步掌握追踪、调试和优化系统行为的核心技能。若您的目标是将创意转化为可靠、可维护的AI产品，本书将全程为您提供指引。

本书的组织结构：路线图

本书分为5个部分，共14章。每个部分都建立在前一部分的基础上，从基础的LLM技能逐步过渡到使用LangChain、LangGraph和MCP构建可用于生产环境的智能体。您可以从头到尾阅读，也可以专注于与您的项目最相关的章节一全文中的交叉引用将帮助您弥合任何知识缺口：

- *第一部分：大语言模型入门（第1-2章）*一本部分奠定基础，探讨大语言模型的优势与局限，以及为何框架对实际应用至关重要。您将了解主要应用模式（引擎、聊天机器人、智能体）和稳健系统的支柱一提示工程与检索增强生成（RAG）：
  - 第1章描绘大语言模型应用的全景，从摘要生成与语义搜索到聊天机器人和智能体，并审视常见挑战。您将看到LangChain、LangGraph和LangSmith如何提供模块化构建块，以及为何提示工程与RAG是构建可靠系统的关键。
  - 第2章提供提示设计的实践入门，涵盖角色设定、上下文、指令、输入和示例，以及单样本/少样本学习策略和思维链（CoT）提示。您将使用LangChain的`PromptTemplate`和`FewShotPromptTemplate`以及OpenAI API来自动生成、测试和迭代提示。

- *第二部分：摘要生成（第3-5章）*—本部分聚焦于大语言模型的一个基础用例：将大量文本提炼为可操作的见解，同时管理上下文限制并为智能体工作流做准备：
  - 第3章为单个大型文档、多文档语料库和结构化数据构建摘要链。您将把源数据加载到`Document`对象中，比较MapReduce与精炼策略，并构建LangChain表达式语言（LCEL）管道，以在不超出上下文限制的前提下保留关键思想。
  - 第4章开发一个研究摘要引擎，它能搜索网络、检索来源、进行摘要并生成简洁、可论证的报告。该过程被分解为子链（搜索、获取、摘要、组合）并通过LCEL进行编排。
  - 第5章从线性链过渡到LangGraph，引入显式状态、节点和边。您将实现条件分支（例如，当证据不足时重新查询）并重构研究助手，以提升其可靠性和可调试性。
- *第三部分：问答聊天机器人（第6-7章）*—本部分将焦点从总结信息转向回答用户问题，揭开RAG的神秘面纱，并为您的系统配备可观测性工具：
  - 第6章使用OpenAI和ChromaDB从基本原理实现RAG，涵盖数据摄取、嵌入、语义搜索以及一个演示摄取-查询循环及向量存储与检索器角色的最小问答聊天机器人。
  - 第7章使用LangChain的RAG组件重建聊天机器人，并集成LangSmith追踪。您将使用加载器、转换器、向量存储、检索器、聊天记忆以及用于简化问答的专用工具。
- *第四部分：高级RAG（第8-10章）*—本部分探讨随着数据和用户规模增长，如何扩展检索能力。您将优化索引和查询策略，并学习组合多个后端：
  - 第8章通过更智能的分块、用于粗粒度与细粒度上下文的多重嵌入以及针对性扩展来增强索引。您将使用`ParentDocumentRetriever`和`MultiVectorRetriever`，为半结构化和多模态内容定制策略。
  - 第9章通过强化查询本身来改进检索：使用重写-检索-读取模式、回退提示、假设文档嵌入（HyDE）和问题分解，以持续呈现更丰富的证据。

  - 第10章将查询路由到合适的后端（向量存储、SQL数据库、文档数据库或知识图谱），生成后端特定的查询（SQL/SPARQL），并使用互逆排序融合（RRF）对结果进行后处理，以向大语言模型提供最相关的上下文。
- *第五部分：AI智能体（第11-14章）*—本部分探讨系统如何做出决策并采取行动。您将构建使用工具和多智能体系统，通过MCP扩展它们，并为生产环境做好准备：
  - 第11章在LangGraph中构建基于工具的智能体，展示如何在模型逐步选择行动时注册和调用工具。您将从单一工具的旅行助手演进到多工具智能体，并在LangSmith中实现完整的可观测性。
  - 第12章使用路由器和监督者模式组合多智能体系统，确保由合适的专家处理每个子任务。您将把智能体连接到真实数据源（SQL/REST），并学习调试和测试跨智能体工作流的实用技术。
  - 第13章介绍模型上下文协议（MCP）：它是什么、为何重要以及如何使用它。您将构建并测试一个MCP服务器（例如天气工具），使用第三方MCP工具，并以清晰安全的方式将远程能力与本地能力集成。
  - 第14章通过添加记忆（通过LangGraph检查点）、分层防护栏和人机协同控制，以及建立持续评估流程以长期保持可靠性，为智能体投入生产环境做好准备。

如何阅读本书

如果您是大语言模型应用的新手，请按顺序阅读第1至第3部分，然后再探索第4部分中更高级的主题。如果您的主要兴趣在于智能体，可以快速浏览第1章和第2章以了解术语，完成第3至第7章中的摘要和RAG基础部分，然后专注于第5部分。每章都包含可运行的示例和后续步骤建议，使您能够逐步将所学内容融入实际项目中。

关于代码

本书包含许多源代码示例，既有编号的代码清单，也有与正文内容穿插的代码。在这两种情况下，源代码都以固定宽度字体显示，以便与普通文本区分开来。有时代码还会被**加粗显示**，以突出本章中相对于之前步骤有所改动的代码，例如在现有代码行中添加新功能时。

在许多情况下，原始源代码已经过重新格式化；我们添加了换行符并调整了缩进，以适应书中可用的页面空间。在极少数情况下，即使这样也不够，因此代码清单中包含了续行标记（↪）。此外，当代码在正文中已有描述时，源代码中的注释通常已从清单中移除。许多代码清单都附有代码注释，用以突出重要概念。

本书中的所有示例代码均使用Python编写，并可在普通开发者笔记本电脑上运行—无需特殊硬件。在必要时，我会提供创建和激活虚拟环境的简要说明，以及一些常用的shell命令，但重点始终牢牢放在Python上。每一章的完整源代码均可从曼宁出版社网站www.manning.com/books/ai-agents-and-applications获取，并在GitHub上镜像于https://mng.bz/rZlZ，您还可以在那里找到更新和勘误。您可以从本书的liveBook（在线）版本https://livebook.manning.com/book/ai-agents-and-applications获取可执行的代码片段。

部分章节以Jupyter Notebook形式呈现，逐行跟随叙述展开；而其他章节则组织为可端到端运行的独立VS Code项目（如果你偏好Cursor，也可在 Cursor 中运行）。Notebook更侧重于解释与实验，而VS Code项目则聚焦于结构化与可复用性。两种方式均基于相同的底层模块，确保您在切换时获得一致的行为体验。

项目面向现代版本的Python（3.11或更高版本）并使用诸如pip和虚拟环境等标准工具。每个章节目录都包含一个`requirements.txt`文件。当示例依赖于外部服务时—通常是模型API或网络搜索—你需要

通过环境变量或本地的`.env`文件提供你自己的凭据。这些情况在文中均有明确标示。

由于您将构建基于大语言模型的应用程序和智能体，大多数示例会使用OpenAI模型，主要来自GPT-5系列。在运行它们之前，您需要创建一个OpenAI账户并关联一张借记卡或信用卡（书中提供了操作说明）。大多数示例在成本最低的GPT-5-nano模型上即可顺畅运行，完成所有示例的总花费应低于5美元。当然，如果您愿意，也可以自由尝试使用更大的模型。

由于智能体系统本质上是非确定性的，您的结果可能与书中展示的略有不同。在需要可重复性的地方，我会固定随机种子、限制模型温度或包含示例记录，以便您比较输出。请将代码仓库视为本书的动态伴侣一运行它、扩展它，并将其作为您自己智能体和应用程序的基础。

liveBook 讨论论坛

购买*《AI智能体与应用》*一书，即可免费访问liveBook—曼宁出版社的在线阅读平台。通过liveBook的讨论功能，您可以为整本书或特定章节和段落添加评论。您可以轻松地记录个人笔记、提问和回答技术问题，并从作者和其他读者那里获得帮助。要访问论坛，请访问https://livebook.manning.com/book/ai-agents-and-applications/discussion。

曼宁出版社致力于提供一个平台，让读者之间以及读者与作者之间能够进行有意义的对话。然而，这并不意味着作者有特定的参与义务，其参与始终是自愿且无报酬的。我们鼓励您向作者提出具有挑战性的问题，以保持他的参与度。只要本书仍在印刷，论坛及过往讨论的存档都将在出版商的网站上保持可访问状态。

作者简介

罗贝托·因凡特(ROBERTO INFANTE)是一位拥有超过25年经验的软件工程师，主要深耕于金融领域—涵盖投资银行、资产管理公司、经纪商和交易所。他目前负责一家对冲基金资金管理部门的量化开发工作，同时主导多个生成式人工智能项目。罗贝托·因凡特也是《构建以太坊去中心化应用》一书的作者。

关于封面插图

《AI Agents and Applications》封面上的插图是“Djiebedji, ou officier des munitionnaires”，意为“Djiebedji，或一名弹药军官”，取自《土耳其服饰画册》（乔治·阿伦茨收藏，纽约公共图书馆，1808–1826年）。

在那个年代，人们很容易通过衣着判断出他们的居住地、职业或社会地位。曼宁出版社以几个世纪前丰富多彩的地域文化为基础设计书籍封面，以此颂扬计算机行业的创造力和进取精神，这些文化通过类似这样的收藏图片得以重现生机。

第一部分 大语言模型入门

本书开篇为后续内容奠定基础，我们将深入探讨构建大语言模型（LLM）驱动应用的“为什么要构建”与“如何构建”—明确其擅长的领域、面临的局限，以及为什么 LangChain、LangGraph 和 LangSmith 等框架对于打造可靠、实用的真实系统至关重要。在此过程中，你将掌握三大核心架构模式：引擎、聊天机器人与智能体，并深入理解支撑它们运行的关键技术，包括提示词工程与检索增强生成（RAG）。

我们还将聚焦人工智能开发中最关键的日常技能之一：提示词的设计与执行。我们将深入探讨如何为不同类型的任务设计提示词，借助单样本（one-shot）与少样本（few-shot）示例优化提示词效果，并通过 LangChain 的提示词模板与 OpenAI API 实现全流程自动化。到本部分结束时，你将超越初步探索阶段，具备构建目标明确、稳定可靠的大语言模型应用的能力。

1 AI智能体与应用导论

本章涵盖以下内容：

- 构建基于大语言模型的应用所面临的核心挑战
- LangChain的模块化架构与核心组件
- 引擎、聊天机器人与智能体的设计模式
- 提示词工程与检索增强生成(RAG)的基础

诸如GPT、Gemini和Claude等大语言模型（LLMs）已从新兴技术演变为不可或缺的基础设施。这些模型使应用程序能够回答复杂问题、生成个性化内容、总结长篇文档，并协调跨系统的多重操作。近年来，大语言模型更催生了一类新型应用—AI智能体。AI智能体以自然语言为输入，自主判断调用哪些工具或服务，协调多步骤工作流，并以清晰、符合人类习惯的友好方式返回结果。

AI应用与智能体系统往往结构复杂，需高效摄入与管理数据、精准构建提示词、可靠地串联模型调用，并无缝集成外部API与服务。幸运的是，LangChain、LangGraph和LangSmith等框架提供了模块化构建单元，能够消除冗余代码、提升最佳实践，使开发者得以聚焦于应用逻辑本身，而非底层连接的繁琐实现。在本书中，你将学习如何运用最前沿的工具与框架，设计、构建并规模化部署真实的基于大语言模型的应用与智能体系统。

1.1 构建基于大语言模型的应用与智能体

大语言模型（LLMs）在处理自然语言方面表现出色—它们能够理解文本、生成内容，并在需要时精准提取信息。这使得它们广泛适用于多种应用场景，如文本摘要、语言翻译、情感分析、语义搜索、聊天机

器人和代码生成。正因如此，如今大语言模型已深入到教育、医疗、法律和金融等多个领域。

尽管应用场景多种多样，大多数大语言模型（LLM）应用在底层架构上却高度相似：它们接收自然语言输入，处理非结构化数据，从一个或多个来源引入额外上下文，最终将所有信息封装为提示词，交由大模型来处理。从宏观上看，这类系统通常可分为三大类：

- *基于大语言模型的应用或引擎* 指的是实现特定、有限功能的系统(例如摘要生成、搜索或内容生成)。此类引擎通常被启动以产生输出，随后便停止；它不会决策接下来要做什么，也不会在请求结束后继续推进工作。
- *聊天机器人* 作为对话式交互接口，能够跨多轮对话维持上下文连贯性。其核心聚焦于自然对话与互动式问答，而非独立执行外部动作。
- *AI智能体*指的是具备自主性或半自主性的系统，利用大语言模型来选择行动、制定计划并执行多步骤任务以达成目标。与引擎不同，智能体可形成(观察 → 决策 → 行动 → 观察)的闭环，协调多个工具或API，并依据中间结果、错误反馈或新信息动态调整后续行为。

在深入探讨LangChain如何支持这三类系统的开发之前，我们将逐一剖析上述每种类型，首先从基于大语言模型的应用与引擎开始。

1.1.1基于大语言模型的应用：摘要生成与问答引擎

基于大语言模型的应用作为后端工具，专为其他系统处理特定的自然语言请求。例如，摘要生成引擎能够将冗长的文本段落压缩为精炼的摘要；这些摘要可立即返回给客户端，也可存入数据库，供其他应用程序后续使用，如图1.1所示。

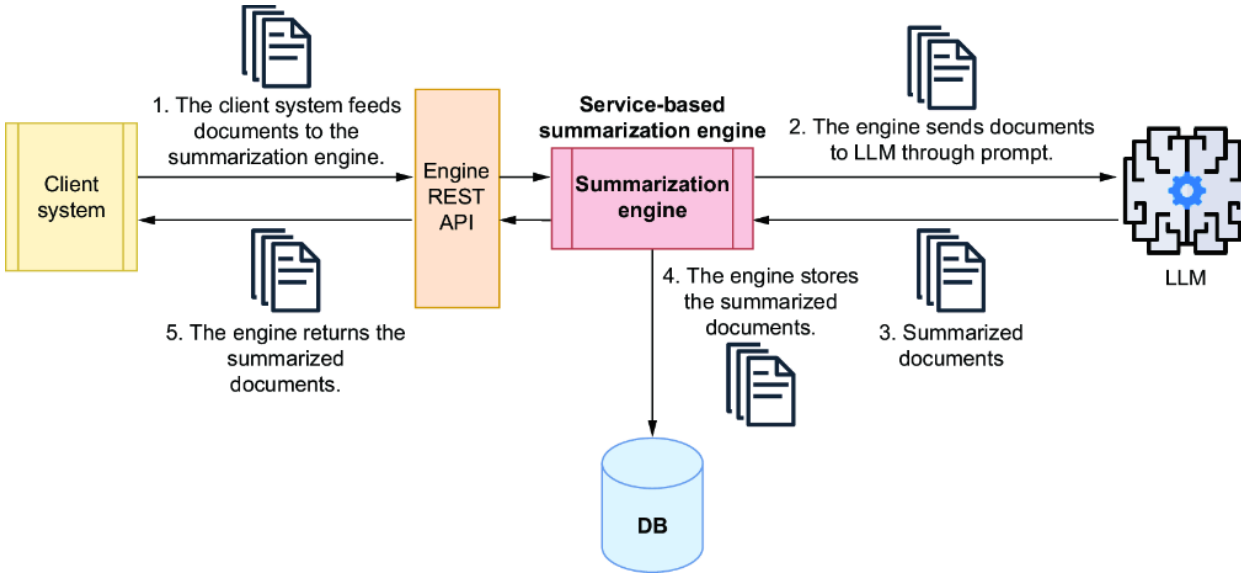

图 1.1 摘要生成引擎能够高效地从海量文本中提取并压缩大量文本内容，同时将生成的摘要进行存储，且可通过 REST API 被其他系统远程调用。

摘要生成引擎通常作为共享服务部署，通过 REST API 向多个系统按需提供能力。另一种常见类型是问答（Q&A）引擎，它基于知识库响应用户查询。问答引擎的工作分为两个阶段：数据摄入与查询。

在内容摄入阶段，引擎通过提取文本、将其切分为片段，并将这些片段转换为嵌入向量—即能够捕捉语义的数学向量—来构建其知识库。这些嵌入向量与原始文本片段一同存储于向量存储中，以实现高效检索。若你对*嵌入模型*或*向量存储*等术语尚不熟悉，无需担忧，后续将详细讲解。现阶段，只需将此步骤理解为：将原始文本转化为可搜索的语义表示形式。

> **定义** *嵌入*是单词、标记或更大文本单元（如句子、段落或文档块）的向量表示，它们被映射到一个连续的高维空间（通常具有数百或数千个维度）。这些向量捕获了语义和句法关系，使语言模型能够理解含义、上下文和相似性。嵌入通常在预训练阶段学习，模型在大规模文本语料库上进行训练，以根据周围上下文预测标记。通过编码单个单词和更广泛的语义含义，嵌入实现了更有效的推理、检索和语言理解。

在查询阶段，引擎接收用户的问题，使用相同的模型将其转换为嵌入向量，并在向量存储库中进行语义搜索。它检索最相关的文本块，并将其与原始问题结合形成提示，随后发送给大语言模型。模型接着利用问题和检索到的上下文生成答案，旨在确保答案准确且基于所提供的来源—尽管结果质量最终取决于检索相关性、分块策略以及模型行为等因素。

这种工作流程被称为检索增强生成（RAG），如图1.2所示。RAG已迅速成为现代大语言模型应用的基石。它是一种基础技术，旨在使大语言模型的输出更有用、更及时，并基于相关的外部信息。

> **定义** *检索增强生成*（RAG)是一种设计模式，其中大语言模型的文本生成通过融入额外的上下文得到增强，这些上下文在查询时从本地知识库（通常存储在向量存储中）检索而来。

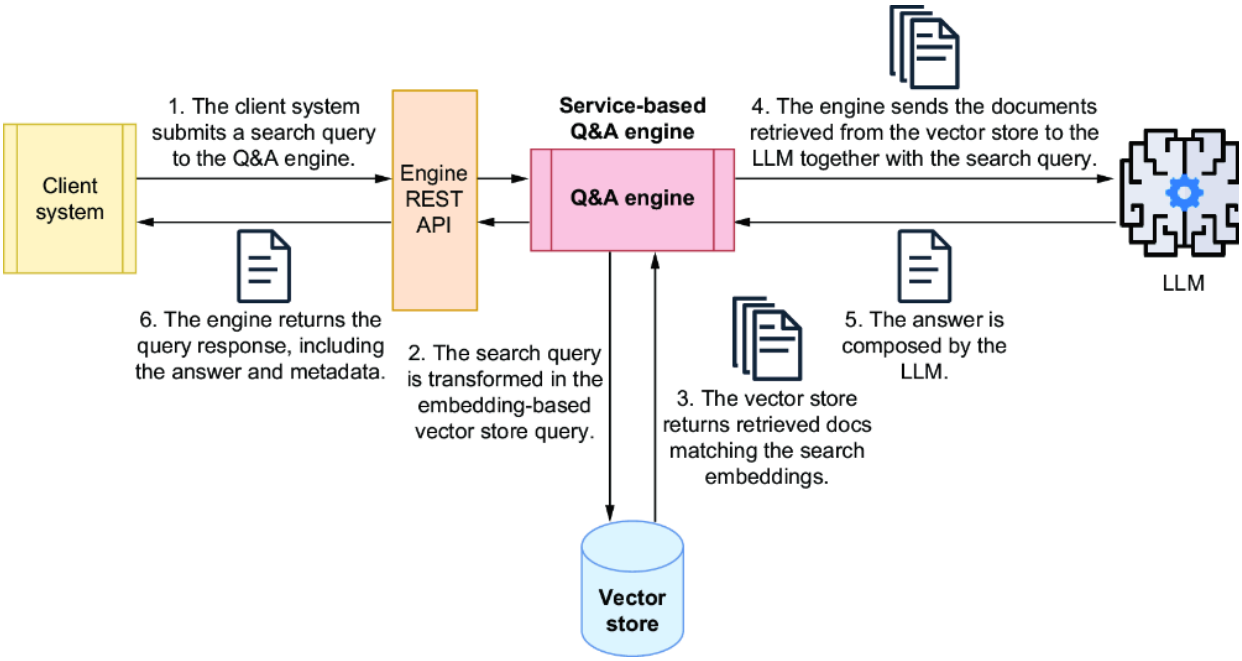

图 1.2 采用RAG设计实现的问答引擎。一个大语言模型查询引擎将特定领域的文档信息存储在向量数据库中。当外部系统发送查询时，它会将自然语言问题转换为其嵌入（或向量）表示，从向量数据库中检索相关文档，然后为大语言模型提供所需信息以生成自然语言响应。

引擎不仅限于问答功能。它们还能通过执行预定义的步骤序列（通常称为链）来调用外部工具。例如，处理用户请求的引擎可能会将自然

语言指令转换为API调用，从外部系统提取数据，然后利用大语言模型来解读结果，并以清晰、易于人类阅读的格式呈现。

这些引擎的核心设计目标是用于系统级工作：自动化流程、智能处理数据以及整合不同平台。它们通过处理那些繁琐的部分—检索、转换、编排—来简化涉及自然语言的工作流，从而让您无需亲自操劳。

稍后我们将看到，AI智能体基于这些能力构建，不过"决策制定"的所在位置会有所不同。引擎通常执行预定义的链条：步骤和分支逻辑大多预先设计，系统针对给定请求运行该工作流直至完成。相比之下，智能体则利用大语言模型在运行时规划和调整其方法—选择调用哪些工具、根据中间结果修订其计划，并可能循环执行多个操作，直到达到停止条件。简而言之：引擎运行工作流，而智能体管理工作流，在开放式的多步骤任务中，以牺牲部分可预测性为代价换取更大的灵活性。

一旦人类开始直接与系统进行实时交互，游戏规则就改变了。此时，引擎承担起对话角色，从纯粹的自动化转向对话交流。这正是聊天机器人发挥作用的地方。

1.1.2 基于大语言模型的聊天机器人

基于大语言模型的聊天机器人扮演着智能助手的角色，能够与语言模型进行持续、自然的对话。与简单的问答脚本不同，这类系统旨在确保交互既实用又安全。它们严重依赖于提示词设计：清晰的指令塑造模型的行为，并有助于防止无关、不准确或不安全的回复。现代聊天API—例如OpenAI所提供的那些—支持基于角色的消息格式（系统、用户、助手），它允许您定义助手的个性特征，并在整个对话中强制维持一致的行为。

为了提高准确性，聊天机器人通常会从本地知识源（如向量数据库）中提取事实性上下文。这使得它们能够将对话流畅性与特定领域的基础知识相结合，从而确保回答不仅流畅，而且相关又可靠。

聊天机器人的一个关键优势在于对话记忆。通过追踪先前的对话轮次，它们能够保持响应的连贯性和个性化。这种记忆受限于模型的上下文窗口。使用更大的上下文窗口会有所改善，但许多系统仍会压缩或总结对话历史，以保持在限制范围内一同时也是为了控制成本。

基于大语言模型的聊天机器人通常专用于摘要生成、问答或翻译等任务。它们可以直接响应用户输入，也可以结合存储的知识进行处理。例如，摘要聊天机器人的架构（图1.3）建立在基础摘要生成引擎之上，但增加了对话管理和上下文感知层。

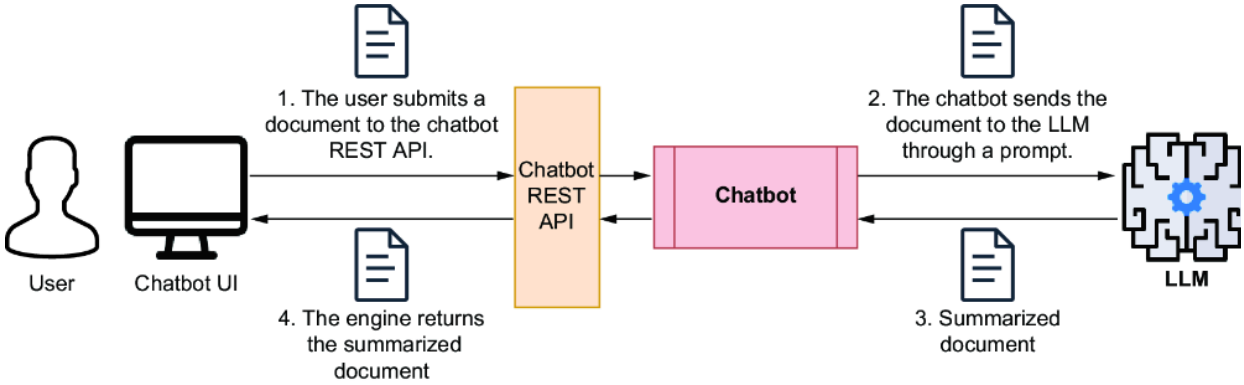

图 1.3 摘要聊天机器人与摘要生成引擎有一些相似之处，但它提供了一种交互式体验，让大语言模型和用户可以共同协作，从而对结果进行微调和改进。

摘要生成引擎与摘要聊天机器人的关键区别在于交互性。聊天机器人允许你实时优化响应：如果你想要更简短或更随意的摘要，只需提出要求即可，如图1.4中的时序图所示。

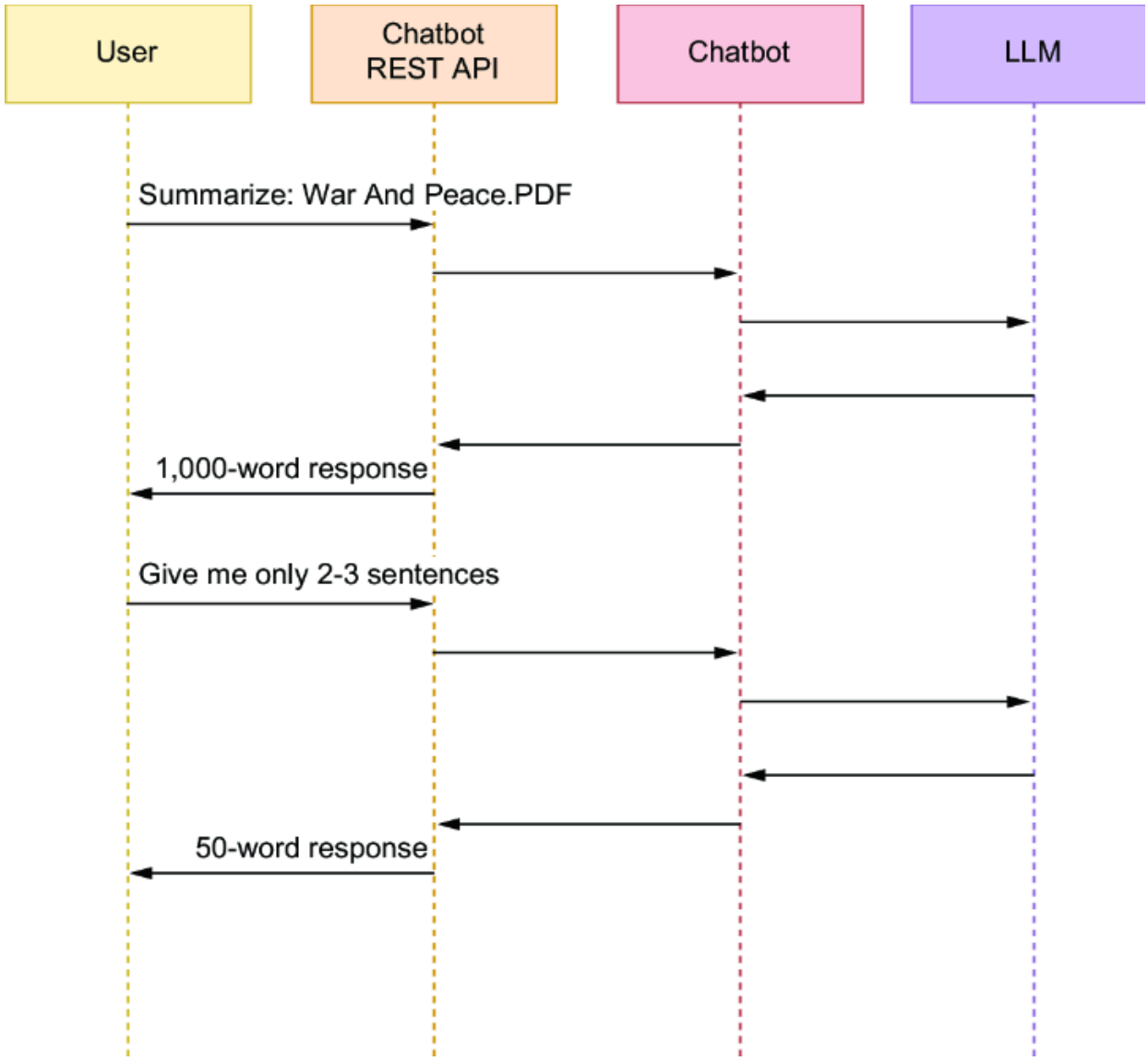

图 1.4 展示用户如何通过聊天机器人与大语言模型交互以生成更简洁摘要的时序图

这种一来一回的互动使得整个过程具有协作性一从而产生更贴合需求、更具情境感知的答案。在下一章中，我们将深入探讨角色指令、少样本示例以及高级提示词工程，让您能更好地控制聊天机器人的行为和输出。

1.1.3 AI智能体

AI智能体是一种与大语言模型协同工作的系统，用于执行多步骤任务一通常涉及多个数据源、分支逻辑和自适应决策。与简单的流水线不

同，智能体具备一定程度的自主性：它们遵循您设定的约束条件，但会自主决定下一步该做什么。

在每个步骤中，智能体都会咨询大语言模型来决定使用哪些工具，然后运行这些工具，并在继续之前处理结果。这个循环会持续进行，直到智能体产生一个完整的解决方案。在实战中，这可能意味着从结构化来源（例如，数据库或API）和非结构化来源（例如，文档或网页）中提取信息，合并结果，并以连贯的格式呈现它们。

考虑以下示例：一家旅行社使用AI智能体根据预订网站的自然语言请求生成度假套餐。如图1.5所示，该流程可能如下所示：

1. AI智能体会向大语言模型发送提示，要求其从可用工具中选出与请求最相关的选项，例如航班、酒店和租车服务商，天气预报服务，以及内部度假优惠数据库。
2. 在开发者精心设计的提示词引导下，该提示词列出了所有可用工具及其功能描述，大语言模型据此选择合适的工具，并生成相应的查询指令—如针对度假优惠数据库的SQL语句，或对外部服务商的REST API调用。
3. 智能体随后执行这些查询，收集返回结果，并将原始度假请求与所获数据一并作为新提示词发送给大语言模型。
4. 大语言模型据此生成一份整合所有预订信息的总结性假期计划，并由智能体将该计划返回至预订网站。

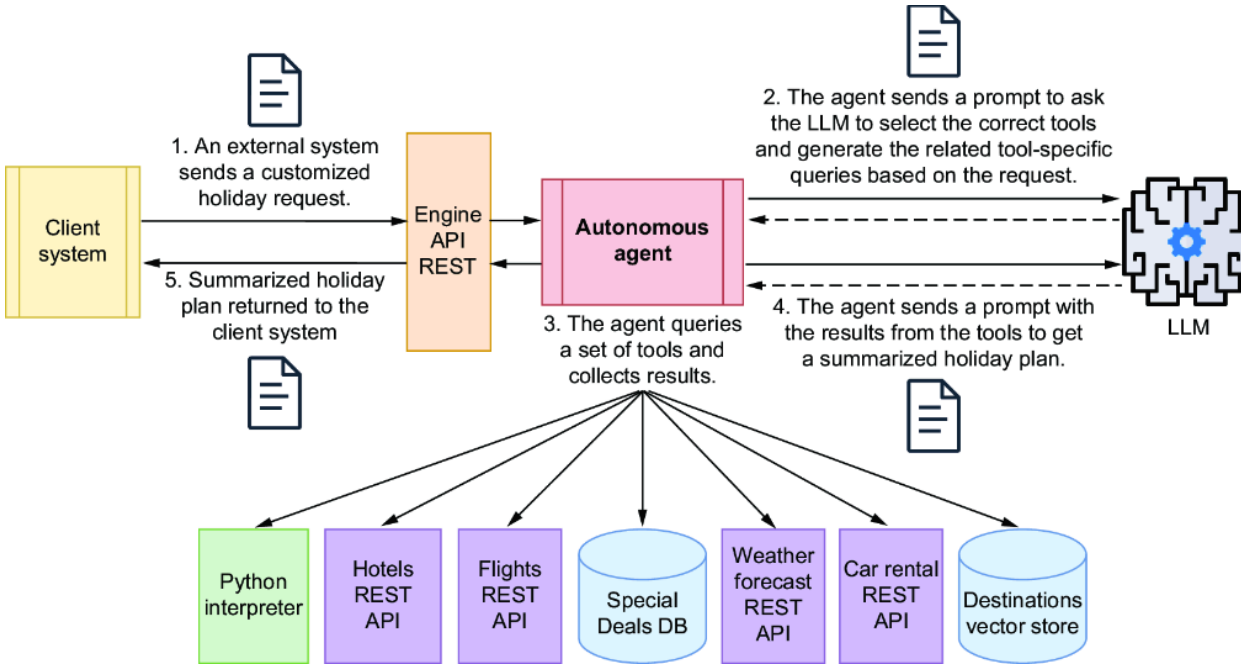

图 1.5 负责组装度假套餐的AI智能体工作流

在生成最终输出（例如一份完整的假期计划）之前，工作流可能涉及智能体与大语言模型之间的多次迭代。此外，图1.5所示的架构仅是一种可能性。另一种可选设计方案可以基于一组更细粒度的智能体，并由顶层的监督者智能体进行协调。

在高风险领域，如金融或医疗，通常包含一个人工介入步骤。这确保了在最终确定关键行动之前，人类可以审查和验证这些行动—例如批准金融交易或确认复杂的医疗建议。在假期规划示例中，可以编程让智能体在将拟定行程发送给客户之前暂停并请求人工批准，从而增加一层额外的监督和信任。

AI智能体领域关注度激增，无论是OpenAI、谷歌、亚马逊等主要参与者，还是LangChain、LlamaIndex、Pydantic等独立开发者，都纷纷发布智能体SDK以推动该技术的更广泛应用。

在本书中，我们将重点介绍LangGraph，它是LangChain的专用智能体框架。LangGraph提供了预构建的智能体和编排器类，以及开箱即用的工具集成，因此您无需重复造轮子。您还可以通过LangGraph获

得构建高级智能体的实战经验，以及学习如何在实战中设计、编排和优化它们。

在许多方面，AI智能体代表了基于大语言模型应用的最先进形式。它充分利用大语言模型的全方位能力—理解、生成和文本推理—来驱动复杂的自动化工作流程。智能体能够根据您设计的提示词，动态选择并使用多种工具，这使得它们在金融、医疗和物流等领域极具价值，因为多步推理和数据整合在这些领域至关重要。

随着Anthropic推出模型上下文协议（MCP），智能体的关注度进一步激增。MCP定义了一项标准，使服务能够通过MCP服务器暴露一些工具，智能体可以通过MCP客户端访问这些工具，其便捷程度如同访问本地组件。这将集成负担转移到了服务本身，使开发者能够专注于构建强大的智能体，而非维护自定义连接器。自2024年末发布以来，MCP迅速成为事实标准—已被OpenAI和Google等主要大语言模型提供商采纳—如今已有数以千计的工具可通过公共MCP门户访问。在本书中，您将学习该协议的工作机制，在实战中考察其生态系统，并构建您自己的MCP服务器，使其可以直接与智能体应用程序集成。

1.2 LangChain 简介

想象一下，你被要求构建一个能够根据公司文档回答客户问题的聊天机器人，或者一个能从数千份内部报告中检索精确答案的搜索引擎。很快你就会发现如下几个相同的痛点：

- 如何在不粘贴完整文档的前提下，可靠地将自有数据注入模型？
- 如何在功能扩展时，保持提示词、链式流程与集成的可维护性？
- 如何在控制上下文长度与计算成本的同时，仍能维持模型输出的准确性？
- 如何在不依赖脆弱胶水代码的前提下，编排多步骤工作流与API调用？
- 应用上线后，如何有效评估、调试与监控其运行行为？

在没有框架可用的情况下，开发者最终会重新实现相同的管道加载数据，然后分割、嵌入、存储、检索并构造提示词一遍又一遍。这种重复正是导致难以可靠地将专有数据引入模型的原因，既避免了提示词信息过载，也防止提示词、链和集成变成难以维护的混乱状态。您还需要在上下文限制、成本与答案质量之间取得平衡。LangChain通过提供一套一致的构建模块来解决这些问题一加载器用于摄取数据，分割器用于分块，嵌入模型与向量存储用于索引数据，以及检索器在查询时仅提取最相关的上下文一这样您就不必将整个文档粘贴到提示词中。提示词模板和结构化的链组合帮助您在功能扩展时标准化和重用提示词与集成，而不是在代码库之间重复代码逻辑。对于多步骤工作流，LCEL和`Runnable`接口为您提供了一种统一的方式来编排工具调用和处理步骤，一旦应用程序上线，其更清晰的结构有助于追踪、调试和评估行为。

LangChain 也在活跃的开源社区推动下迅猛发展。它持续跟进大语言模型架构、新数据源和检索技术的的前沿进展，同时助力建立行业共享的最佳实践，以构建和部署基于大语言模型系统。

LangChain的设计遵循三大原则：模块化、可组合性和可扩展性。组件遵循标准接口，因此您可以更换大语言模型、调整向量存储或添加新的数据连接器，而无需重写整个应用程序。真实场景中任务可以通过组合多个组件来构建，形成能够动态选择合适工具的链式流程或智能体工作流。虽然系统提供了默认实现，但您始终可以用自定义逻辑或第三方集成来扩展或替换它们，从而避免厂商锁定并提升系统的互操作性。

通过学习LangChain，你不仅能获得构建生产级大语言模型应用的能力，还能掌握可迁移的技能。同类竞品框架往往以相似的方式解决相似的问题，因此一旦你理解了这些模式，未来无论选择何种技术栈，你都能灵活运用这些知识。

1.2.1 LangChain 架构

LangChain的文档非常详尽，但理解其工作原理的最佳方式是通过实际构建来学习。本节为您提供一个宏观概述，我们将在后续章节中反复提及。您可以将其视为一张地图，在我们深入探讨具体细节和代码示例时，它将为您指引方向。

大多数大语言模型框架遵循相似的整体模式，但每个框架都定义了各自的组件。在LangChain中，工作流程大致如图1.6所示。首先从不同来源（文件、数据库或网站）提取文本，并将其包装成`Document`对象。这些文档通常被分割成更小的块，以便更容易处理。接下来，每个块通过一个嵌入模型，将文本转换为捕捉其含义的向量。原始块及其嵌入向量都存储在一个向量存储中，这使您能够基于相似性搜索快速检索最相关的文本片段。

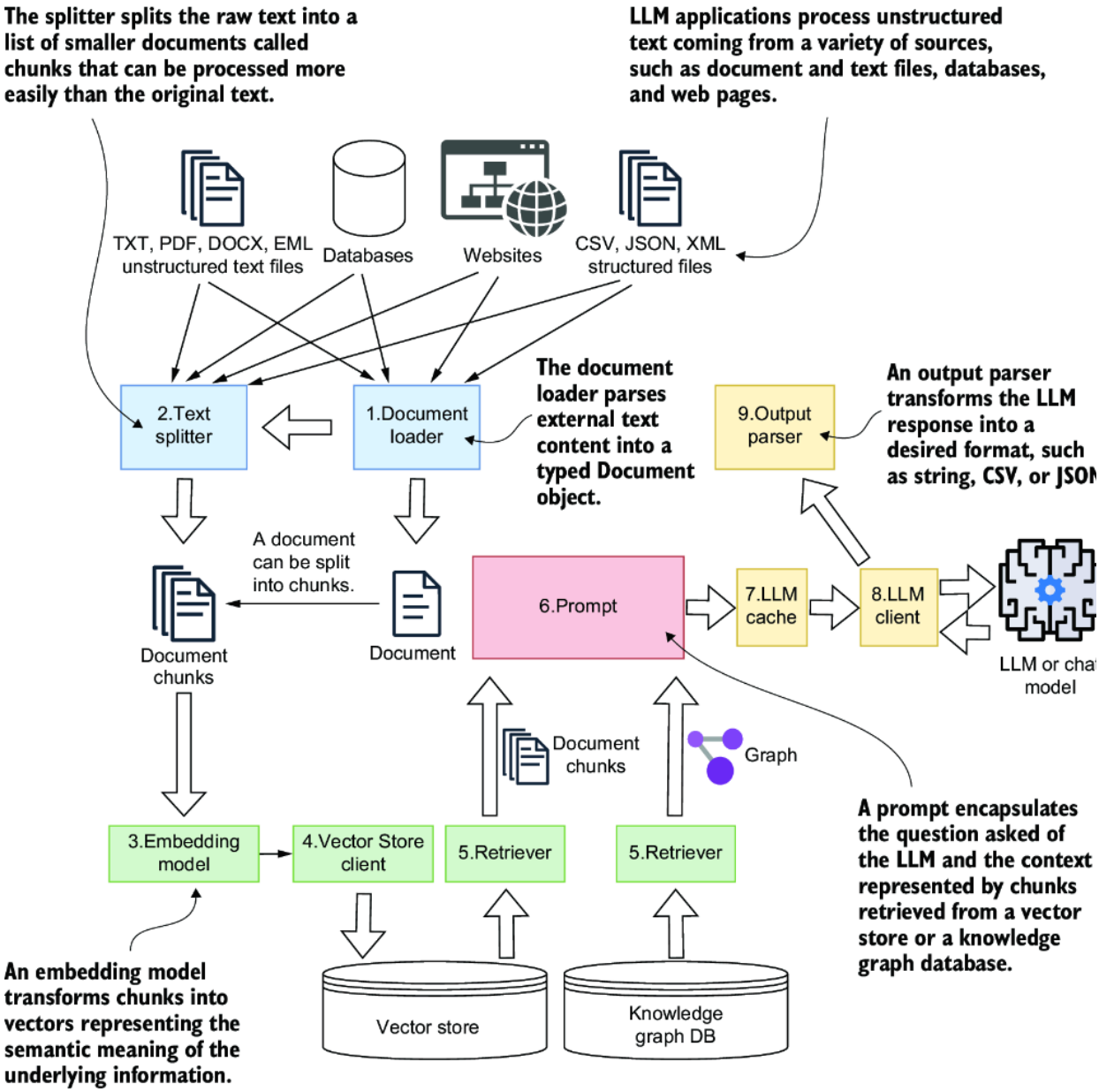

图 1.6 LangChain 架构。文档加载器导入数据，文本分割器将其分割成块。这些块通过嵌入模型进行向量化，存储在向量数据库中，并通过检索器为大语言模型检索相关文档片段。大语言模型 检查先前的请求对应的缓存以返回缓存的响应，而输出解析器则格式化大语言模型的最终响应。

当大语言模型应用程序执行任务时一例如摘要生成或语义搜索一它会构建一个提示词，将用户的问题与额外上下文结合起来。该上下文通常来自从向量存储中提取的文档片段。然而，有时您可能还需要从图数据库中引入信息。向量存储仍然是大多数RAG工作流程的基石，但图数据库在需要表示和推理实体之间关系的应用程序中变得越来越常

见。这一趋势在智能体架构中尤为关键：与引擎（通常执行单个检索步骤并返回输出）不同，智能体可能需要维护更持久的记忆，持续追踪实体及其随时间演变的关系，并使用这些关系规划后续行动（例如，决定调用哪个工具或接下来检索哪些信息）。LangChain已经原生集成Neo4j等流行图数据库，并允许您使用基于图的记忆或规划组件—这些功能在高级智能体架构中越来越常见。

为了让所有这些组件更容易连接在一起，LangChain引入了`Runnable`接口和LCEL。它们提供了一种简洁、一致的方式来链式组合组件，而无需编写大量胶水代码。我们将在本章后续部分更深入地探讨这两者。

同样值得记住的是，LangChain的工作流程并不局限于简单的管道。你可以将组件编排成图来处理更复杂的分支流程—这一能力在LangGraph模块中得到了正式实现。我们将在下一节深入探讨这些模式和架构变体，届时大语言模型应用程序设计的宏观图景将变得清晰。

以下是每个组件的详细说明（参考图1.6中的编号）：

- *文档加载器（Document Loaders）（1)*— 从多种数据源（文件、网页、SaaS 工具、数据库）中提取内容，并转换为LangChain 的 `Document` 对象。
- *文本分割器（Text Splitters）（2)*— 将大段文本拆分为较小的文档(`Document`)片段(Chunk)，使其适配模型上下文窗口，便于后续嵌入、索引与高效检索。
- 文档对象(`Document`)— LangChain 的核心数据容器，用于封装原始内容与元数据（如来源、页码、作者、时间戳），贯穿从数据摄入到检索的全流程。
- *嵌入模型（Embedding Models）（3)*— 将文本转化为高维向量，捕捉语义，实现基于语义相似度的检索与匹配。
- *向量存储（Vector Stores）（4)*— 专为存储嵌入向量(及其关联的 `Document` 对象)而设计的数据库，支持RAG高效语义检索。

- *知识图谱数据库（Knowledge Graph Databases）*— 图数据库，用于存储实体（Entity）及其关系（Relationship），适用于需建模并查询概念间关联而非仅文本相似性的场景。
- *检索器（Retrievers）（5)*— 用于查询一个或多个后端（向量存储、SQL 数据库、图数据库），返回与用户问题最相关的 Document 集合的组件。
- *提示词模板（Prompts）（6)*— 可复用的提示词模板，将用户输入与检索到的上下文（以及可选的示例）组合，构造成发送给大语言模型的最终请求。
- *大语言模型缓存（LLM Cache）（7)*— 可选层，缓存历史响应，对重复或语义相似的查询直接返回缓存结果，显著降低响应延迟与调用成本。
- *大语言模型/聊天模型（LLM/Chat Model）（8)*— 用于生成输出的大语言模型接口；LangChain支持多个提供者，还支持用于测试的模拟/伪模型。
- *输出解析器（Output Parser）（9)*— 将大语言模型的响应转换为结构化格式（如 JSON），确保下游系统可稳定处理。

上述列表中的组件可以组织成链式结构，也可以围绕智能体进行构建：

- *链（Chain）*——一种组合结构，用于引导 LangChain 的处理流程，针对特定应用场景定制，并基于所描述组件的有序序列构建。
- *智能体（Agent）*— 该组件通过扩展了一个顺序链来管理一个动态工作流，智能体的处理过程具有灵活性，且能够根据用户输入或组件输出来进行调整。在大多数框架的文档中，动态工作流中的资源被称为*工具*或*插件*，所有工具的集合则被称为*工具包*。

LangChain的全面设计支持三种主要应用类型，我们稍后将详细探讨：摘要生成与查询服务、聊天机器人以及智能体。尽管该框架乍看之下可能显得复杂，但通过逐步剖析示例并构建各个组件，其结构和核心概念将逐渐清晰。

1.2.2 LangChain的核心对象模型

在充分理解LangChain的宏观架构之后，我们现在可以转向其核心对象模型，以便更清晰地了解该框架的运作机制。理解其中的关键对象及其交互方式，将显著提升您有效使用LangChain的能力。同样重要的是，这些类族反映了您在大多数大语言模型应用中会遇到的基础构建模块—文档、分块、检索、提示词和模型调用—因此学习LangChain对象模型也能为您提供关于大语言模型应用通常如何组装的实用心智模型。

对象模型被组织成类层次结构，以抽象基类为起点，衍生出多种具体实现。图 1.7 从视觉上概览了各类任务中的核心类族，所有这些类族均围绕`Document`实体构建。它说明了加载器如何生成`Document`对象，分割器如何将其拆分为更小的片段，以及这些片段随后如何被传递到向量存储和检索器中进行下游处理。

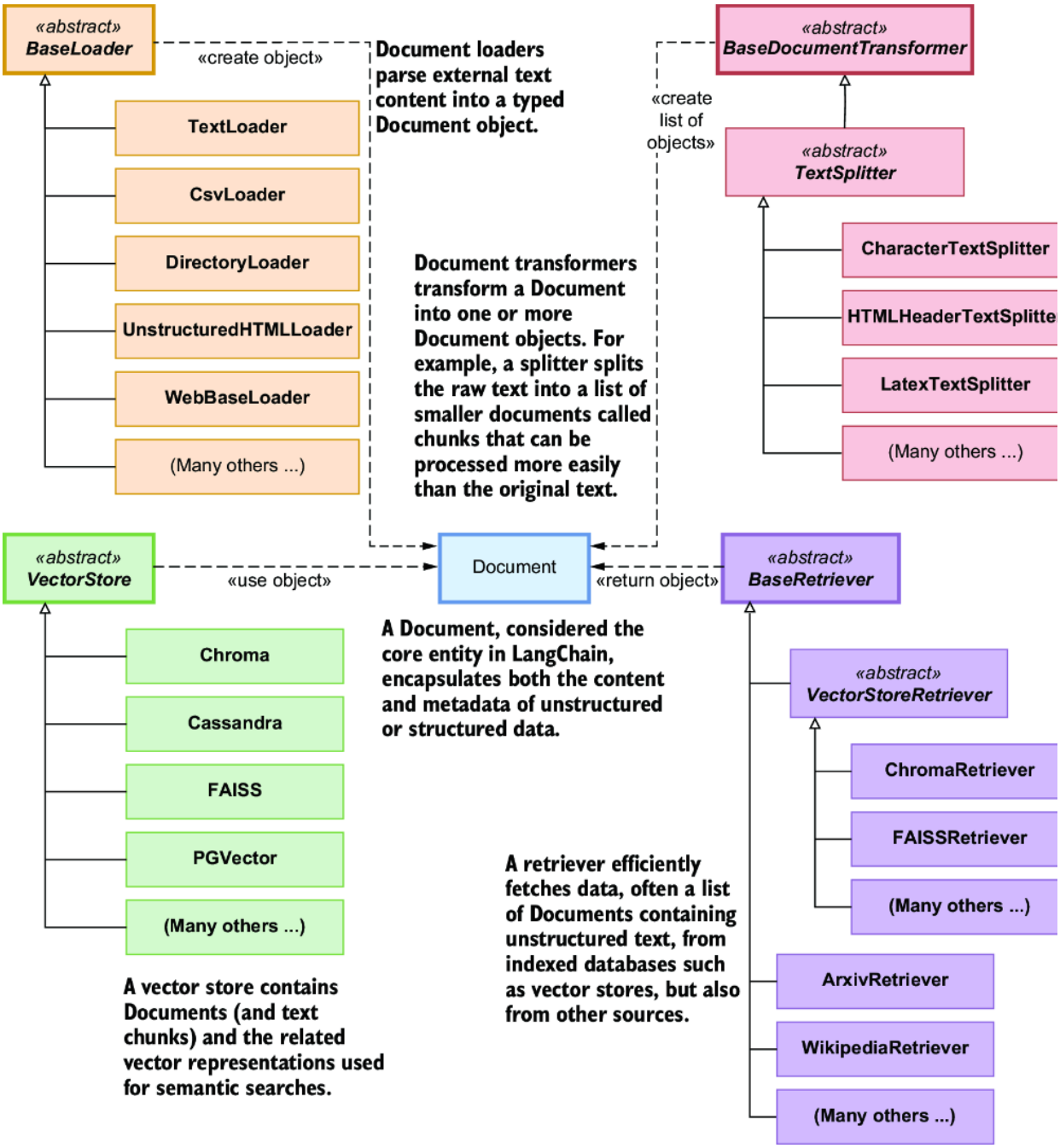

图 1.7与Document核心实体相关的类对象模型，包括Document加载器（创建Document对象）、分割器（创建Document对象列表）、向量存储（在向量存储中存储Document对象）以及检索器（从向量存储和其他来源中检索Document对象）

正如第1.2.1节所述，LangChain 架构中的主要类如下所示：

- `Document`（文档对象）
- `DocumentLoader`（文档加载器）

- `TextSplitter`（文档分割器）
- `VectorStore`（向量存储）
- `Retriever`（文档检索器）

LangChain 在这些组件层面广泛集成多种第三方工具与服务，展现出极高的灵活性。支持的所有集成列表可参阅其官方文档。此外，LangChain 还提供了 LangChain Hub—一个由社区驱动的仓库，用于发现和共享可复用的组件，如提示词模板（prompts）、流程链（chains）与工具（tools）。

许多组件—从加载器到大语言模型（LLM）—共享一个关键的架构特性：`Runnable`接口。这一通用接口使得各类对象能够以一致的方式被组合与链式调用，从而构建出高度模块化的工作流。关于该特性的深入解析，以及对 LCEL（LangChain 表达式语言）的详细探讨，将在后续专门聚焦于构建可组合、富有表现力的大语言模型流水线的章节中展开。

在图1.8中，您可以看到与大语言模型相关的对象模型，包括`PromptTemplate`和`PromptValue`。该图说明了这些类如何连接到大语言模型接口，展示了一个比之前介绍的类更为复杂的层次结构。

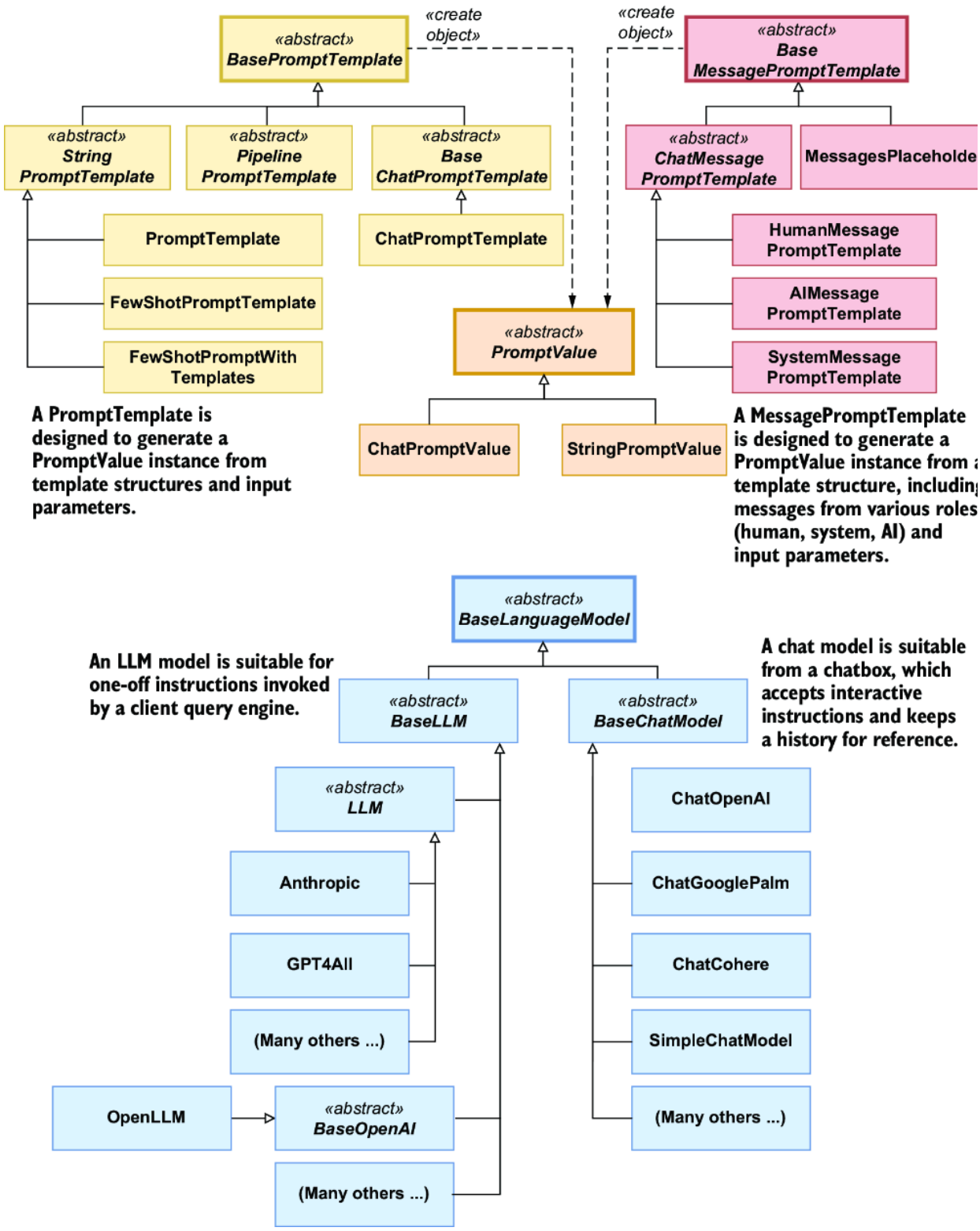

图 1.8 与大语言模型相关的类对象模型，包括 `PromptTemplates` 和 `PromptValues`

既然您已经了解了LangChain的用途、架构以及它所支持的核心应用类型，我鼓励您使用附录B中描述的Jupyter Notebook来尝试LangChain。在我们深入探讨之前，这是一种快速且实用的方式来感受这个框架。

> **注意：** 由于您将构建基于大语言模型的LangChain应用程序和LangGraph智能体，本书中的大多数示例都使用OpenAI模型，主要来自GPT-5系列。在运行这些示例之前，您需要创建一个OpenAI账户并绑定借记卡或信用卡（具体操作说明请参见附录A）。大多数示例在成本最低的GPT-5-nano模型上即可顺畅运行，完成所有练习的总花费应该会低于5美元。当然，如果您愿意的话，也可以尝试使用更大的模型进行实验。

每个LangChain应用的核心都蕴含着大语言模型的强大能力。基于大语言模型的应用可以被视为围绕模型的专用封装器或接口，专为特定应用场景量身定制。接下来，我们将深入探讨大语言模型特别适合的几种主要应用场景。

1.3 大语言模型的典型应用场景

大语言模型被广泛应用于各种任务，涵盖从文本分类到代码生成和逻辑推理。以下是一些最常见的应用场景，以及可供进一步探索的真实案例：

- *文本分类与情感分析*—包括根据情感分析对新闻文章进行分类或推荐股票。例如，GoDaddy 使用大语言模型自动分类客服工单，详见文章《大语言模型实战经验：GoDaddy在模型使用上10条经验教训》(https://mng.bz/8X52).
- *自然语言理解与生成*—大语言模型能够识别文本中的主要主题，并生成符合长度、语气或术语要求的摘要。Duolingo利用AI加速课程创建，详见博客文章《多邻国如何利用AI更快速创建课程》(https://mng.bz/Ewpl).

- *语义搜索*—基于问题的意图与上下文查询知识库，而非依赖简单关键词。Picnic超市App利用大语言模型优化食谱搜索，详见Medium 文章《利用大语言模型增强搜索与检索》(https://mng.bz/NwW2).
- *自主推理与工作流执行*—大语言模型能够通过理解用户需求并管理流程的每个步骤，完成如规划完整度假套餐等复杂任务。
- *结构化数据提取*—指的是从非结构化文本（比如财务报告或新闻文章）中抽取结构化数据-实体及其相互关系
- *代码理解与生成*—大语言模型能够分析代码，识别问题、提出改进建议，或根据用户指令生成新的代码组件—涵盖从简单的函数与类，到完整的应用程序。这一能力为GitHub Copilot、Cline AI 等主流 IDE 插件，以及 Cursor、Windsurf 等新兴 AI 编程助手赋能，它们可在你编码时提供实时的代码建议与错误检测。此外，基于命令行界面（CLI）的编码工具，如 Anthropic 的 Claude Code 和 OpenAI 的 Codex，允许开发者通过命令行终端与 AI 交互，直接实现代码生成、重构与调试。
- *个性化教育与辅导*—大语言模型正日益被用作互动式导师，提供个性化的辅导与反馈。例如，可汗学院(Khan Academy)的Khanmigo 就利用大语言模型辅助学生开展互动式学习。

这些应用场景假设大语言模型能够胜任用户请求。然而，现实中的任务往往涉及超出模型初始训练范围的特定领域或场景。如何确保你的大语言模型在这些情况下仍能有效满足用户需求？这正是下一节将要讲解的内容。

1.4 如何根据您的需求调整大语言模型

你可以采用多种技术手段来提升大语言模型响应用户请求的能力，即使其此前并未接受过特定领域的先验知识或训练，这些技术如下所示。

- 提示词工程

- 检索增强生成 (RAG)
- 微调

让我们从最基础的方法开始：提示词工程。

1.4.1 提示词工程

大语言模型的提示词可以简单到仅包含一条指令，也可以复杂到包含丰富示例和上下文的一整段说明。*提示词工程* 就是设计这些输入内容，以使模型理解任务并生成有用、准确响应的实践。如果做得好，它能让开发者引导模型的行为，使其适应特定领域的需求，甚至处理模型未经过明确训练的问题。这里一个常见的技术是*上下文学习*，模型直接从嵌入在提示词中的示例推断模式—无需进行微调。

在实战中，提示词通常被组织为模板：一个固定的指令，带有接受动态输入的变量字段。这使得提示词在应用程序的不同部分可重用且更易于管理。一种广泛使用的模式是*少样本提示*，即包含少量示例来指导模型如何泛化到类似的输入。

精心设计的提示词具有惊人的强大效果。它们能够引导大语言模型生成高质量、具备领域感知的输出，而无需额外的训练数据。例如，聊天机器人通常会将最近的对话轮次嵌入到提示词中，这样模型就能保持上下文，并生成连贯的多轮回复。提示词工程往往足以应对大量实际应用场景，使其成为许多实际应用中一种轻量级却高效的工具。

LangChain 基于这一理念，通过抽象层来一致地管理和复用提示词，我们将在下一章探讨这一点。话虽如此，仅靠提示词工程有其局限性—尤其当应用需要将回答的答案锚定在用户特定或企业专有数据时。此时，自然的下一步便是 采用 RAG（本章前面已提及），它通过从外部来源动态提取知识来增强提示词。

1.4.2 检索增强生成(RAG)

将大语言模型的响应锚定在你自己的数据上，是提升其回答质量最有效的方法之一。与其仅依赖模型在预训练阶段习得的知识，你可以通过实现 检索增强生成（RAG），从本地知识库动态检索相关上下文（通常存储于向量数据库中），并将其注入提示中。检索增强生成(RAG)已成为现代大语言模型应用中的核心模式。

构建知识库的工作流程始于文档的摄入。文档被加载后，接着被切分为更小的文本块，并通过嵌入模型转换为向量表示。这些向量捕捉了文本的语义内涵，使得系统能够基于语义相似性而非特定文字字面匹配来比较不同文本内容。LangChain 提供了一系列工具，支持以多种格式加载文档、高效切分文本、生成嵌入向量，并将所有内容存储至向量数据库中（参见图 1.9）。

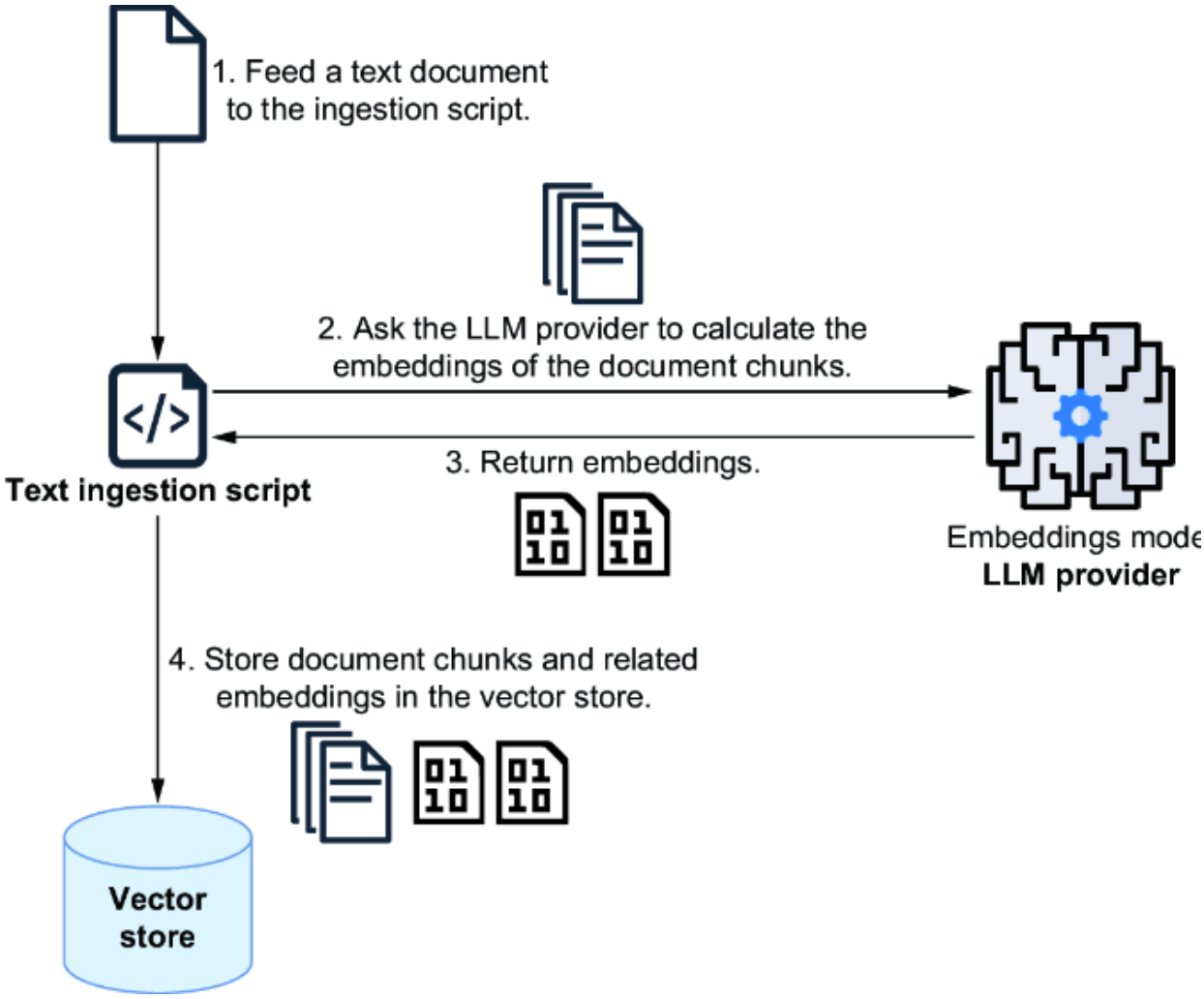

图 1.9 文档集合被切分成文本块并转换为基于向量的嵌入表示。随后，文本块及其相关嵌入向量均被存储于向量数据库中。

检索增强生成(RAG)具有多重优势：

- *高效*—无需将整份文档传入模型，仅检索关键文本块，使输入保持精简，降低token开销，并适配上下文长度限制。
- *精确*—响应*锚定*于真实数据，显著降低了模型产生*幻觉*的风险。本书后续章节将介绍让大语言模型标注信息来源的技术，进一步提升回答的透明度与可信度。
- *灵活*—通过替换嵌入模型、检索器或向量数据库，可将同一架构模式适配至不同领域与应用需求。

定义 *锚定*大语言模型指的是构建提示词，而该提示词一般包括从可信知识源中提取的上下文，这些上下文信息通常又存储于一个向量数据库中。此举确保了模型的输出是基于经过验证的事实，而非仅依赖其预训练知识，因为后者可能包含过时或不可靠的数据。

定义 当大语言模型（LLM）生成了错误、具有误导性或完全虚构的响应时，即发生了*幻觉*。这种情况通常源于模型在训练过程中依赖了低质量数据，或缺乏足够信息以准确作答。由于其自身自回归特性，大语言模型会在缺乏相关上下文时仍试图生成响应-用听起来合理但实际错误的信息填补认知空白，从而导致产生幻觉。

为了使检索增强生成(RAG)系统可靠，提示词应明确指示大语言模型仅依赖检索到的上下文。LangChain还支持防护栏和验证器来帮助强制执行安全行为。在高风险场景中，人工介入审查仍是最佳保障措施。

简而言之，检索增强生成(RAG)填补了静态预训练模型与动态、特定领域应用之间的鸿沟。通过教会你的应用与大语言模型"说同一种语言"一向量一你便解锁了一种提供有事实依据且经济高效答案的实用方法。如果提示词工程和检索增强生成(RAG)仍无法满足你的需求，下一步就是对模型进行微调，我们将在下一节中详细讨论。我们将深入探讨检索增强生成(RAG)，并探索扩展其在真实应用中能力的进阶模式与技巧。

1.4.3 微调

微调指的是通过在精心筛选的示例数据集上对预训练的大语言模型进行训练，使预训练大语言模型在特定任务或领域表现更佳的过程。这些示例包含了您希望模型掌握的特定风格、术语和推理模式。从传统意义上来讲，微调需要依赖专业的机器学习框架与高性能计算硬件，但如今许多平台一包括OpenAI的微调API以及多个开源工具包一使得仅需上传数据集并进行最简配置即可实现。

微调的主要优势在于效率：一旦模型吸收了特定领域知识，便无需在每个提示词中冗长地嵌入指令或示例，模型自身"知晓"如何在特定上下文中作出响应。然而，权衡利弊同样切实存在。准备高质量训练数据集需要耗费大量时间与专业知识，而训练过程往往成本高昂，因其通常依赖GPU资源。

诸如低秩适应（LoRA）和其他参数高效微调方法的最新进展，降低了成本和复杂性，使得微调变得更加易于实现。相关的技术，例如指令微调和人类反馈强化学习（RLHF），推动模型更可靠地遵循指令，尽管它们通常需要更多的工程努力和基础设施。

近些年来，低秩适应（LoRA）和其他参数高效微调方法的最新进展显著降低了微调的成本与复杂性，使得微调技术更加普及。与此同时，相关技术如指令调优和*基于人类反馈的强化学习(RLHF)*进一步推动了模型更可靠地遵循指令，但通常需要更复杂的工程实现与更雄厚的基础设施支持。

微调是否真正必要是一个持续存在的争论。通用大语言模型在开箱即用时就展现出惊人的强大能力，尤其是在与检索增强生成(RAG)技术结合使用时。事实上，在《微调 vs. RAG：冷门知识场景下》（作者：海达尔·苏达尼(Heydar Soudani)、埃万杰洛斯·卡努拉斯(Evangelos Kanoulas)和法格赫·哈西比(Faegheh Hasibi); https://arxiv.org/abs/2403.01432）中的研究表明，检索增强生成(RAG)通常优于微调，因为它通过让你在运行时动态地提供上下文，降低了成本，并减少了重新训练的需求。

尽管如此，在高度专业化的领域一例如医学、法律或金融一微调仍然具有不可估量的价值。它使得模型能够捕捉到特定领域的术语和工作流程，这是通用模型难以企及的。众所周知的应用案例如下所示：

- BioMistral (生物学与生命科学领域)
- LexiGPT (LexisNexis开发的法律领域大语言模型)
- BloombergGPT (金融领域专用模型)
- Anthropic 的 Claude Code以及其他专注于代码生成的模型

总而言之，微调通过定制化大语言模型来获得领域专业知识和特定精度，但这是以耗费时间、金钱和复杂性为代价的。作为开发者，您需要权衡何时真正值得进行微调，以及何时使用检索增强生成(RAG)或提示词工程能更快达成目标。话虽如此，本书并不会涵盖大语言模型微调内容，因为我们的重点是构建AI智能体和应用程序—而非创建或修改模型本身。

1.5 选择哪种大语言模型

在开发基于大语言模型应用时，你会发现有众多模型可供选择。有些是专有模型，需要通过订阅或按使用量付费的API来访问；而另一些则是开源的，可以自行托管。大多数现代大语言模型都提供REST API以便于集成，并配有用户友好的聊天界面。许多模型还提供不同规模的变体，让你能在性能、速度和成本之间找到合适的平衡点。

LangChain 使得与不同大语言模型的集成变得简单。这得益于其标准化的接口，您只需进行最少的代码更改即可切换模型—这在当今快速演变的大语言模型领域中是一项至关重要的功能。

选择模型时需要考虑以下三个主要因素：

- *精确度*—通常而言，模型规模越大，其精确度越高，因其在更大规模的数据集上完成训练。然而，在推理阶段，它们的响应速度则更慢，成本也更高，因为厂商对每百万token收取额外费用。
- *速度（响应延迟）*—较小的模型响应速度更快，这使其在对响应时效性要求严苛的交互式应用如聊天机器人中极具吸引力，但代价是精确度相对较低。
- *成本*—推理成本与模型大小直接挂钩。更大、更准确的模型对应的每个token的费用也更高，而规模较大的情况下使用较小的模型则成本更低。

在实战中，你需要在这三个维度之间取得平衡：最“理想”的模型往往并非是最大或最便宜的，而是最契合你的应用程序需求的。例如，客

户服务聊天机器人可能更看重响应速度与部署成本，而法律文书分析工具则会优先追求精确度，即使这意味着承担更高的费用。除了这些核心权衡外，还有其他因素会影响模型选型，而这些因素会使得某一类模型家族在特定场景中更具优势：

- *模型用途*—对于摘要生成、翻译或分类等通用任务，大部分主流模型（如 GPT、Gemini、Claude、Llama、Mistral）均能胜任；针对代码生成等专业化需求，则应选择领域专用的微调模型，例如 Meta 的 Code Llama或 Anthropic 的 Claude Code。
- *上下文窗口大小*—更大的上下文窗口支持处理更长的提示词与文档。部分模型可支持数百万token，而多数标准 API 通常限制在128K 至 256K token之间。扩大窗口大小能显著拓展应用场景，但同时也会增加计算开销与推理成本。
- *多语言支持*—若你的应用需处理多种语言，应优先选择具备强多语言训练背景的模型。Qwen 与 Llama 在语言覆盖广度上表现突出，而部分Gemma 版本则专为亚洲语言优化，具备更优的本地化表现。
- *指令型模型 vs. 推理型模型*—指令型模型（如 GPT-5-mini、Gemini Pro）擅长在明确步骤指引下执行任务；推理型模型（如GPT-5 Thinking、Gemini Thinking）则专为自主拆解问题、规划解决路径而设计。这里的权衡取舍主要表现在精确度/速度/成本这3方面：推理模型能力更强，但响应更慢、成本更高。
- *开源模型 vs. 专有模型*—开源模型（如 Llama、Mistral、Qwen、Falcon）提供数据隐私保障、部署自主权与定制灵活性；另一方面，专有API 则具备更低的接入门槛，通常能以更少工程投入获得业界顶尖性能，但长期使用可能需要高昂费用。

通过权衡这些因素，您可以选择一个符合项目目标和约束条件的大语言模型。在许多应用中，最有效的策略是为不同任务使用不同的模型。例如，一个工作流程可能使用GPT-5-nano进行快速摘要生成，使用GPT-5-mini进行答案合成，并使用完整的GPT-5来处理复杂查询路由—在整个系统中平衡精确度、速度和成本。

1.6 你从本书中能学到什么

如果您已经读到这里，那么您已经认识到构建由大语言模型驱动的应用程序的价值。大语言模型交互正迅速成为现代软件的核心功能一类似于21世纪初网站为基于服务器的应用程序开辟了新的通信渠道。这一转变正在不断加速发展。

我们将从提示词工程开始，这是与大语言模型有效交互的基础技能。您将首先通过ChatGPT进行交互式实验，然后转向通过REST API进行编程式访问。这些练习将为本书中的许多项目奠定基础。

接下来，我们将使用LangChain构建两类应用：定制引擎（例如摘要生成和问答系统）以及将对话流畅性与知识检索相结合的聊天机器人。每个项目虽小，但都五脏俱全，但所有项目都将共享旅游行业这一共同背景主题，以便您能在一个连贯的领域中看到这些理念如何相互契合。

在打好这些基础之后，我们将转向 LangGraph，用于构建 AI智能体一这类应用能够协调多步骤工作流、整合各类工具，并做出自适应决策。这将带你初次接触最前沿的大语言模型驱动系统：引擎与工具在此融合，形成动态且自主运行的应用程序。为降低入门门槛，我将从一个简单的 Python 脚本开始，逐步扩展其功能。每个扩展都将衍生出不同的能力一例如工具调用、规划能力或记忆机制一让你在循序渐进中理解AI智能体如何逐步提升复杂度，而不会因一上来就难度过大而被击溃。

我们还将深入探讨检索增强生成(RAG)，这是许多现实世界系统背后的架构模式。我将通过简短、聚焦的脚本来介绍检索增强生成(RAG)概念一这些脚本有些是独立的，有些则逐步精炼为更高级的工作流程一这样您就能同时掌握基础知识和前沿技术。

尽管示例中引用了OpenAI模型以便于理解和快速上手，您还将学习如何通过附录E中列出的推理引擎来使用开源模型。通过这种方式，您将

掌握构建和部署应用程序的技能，这些应用程序不仅成本效益高、注重隐私保护，而且完全处于您的掌控之下。

除了构建之外，您还将探索大语言模型应用的完整生命周期：使用LangSmith进行调试、监控和优化；通过LangGraph编排复杂的工作流；并应用生产部署的最佳实践，以确保可扩展性和可维护性。

读完本书后，你将能构建出一系列可运行的应用项目，掌握核心架构模式，并培养出能自信地设计和实现基于大语言模型系统的能力。你将不仅能理解这些应用的工作原理一更能时刻准备着持续创新，不断拓展LangChain、LangGraph与大语言模型所能实现的边界。

总结

- 基于大语言模型的系统主要分为三类：引擎（执行特定任务，如摘要生成）、 聊天机器人（具备记忆能力的双向对话系统）和智能体（能自主选择工具并执行多步骤任务的自治系统）。
- 嵌入（Embeddings）将文本转化为能够捕捉语义的数学向量，文本片段及其对应的嵌入向量均被存储于向量数据库中，以实现基于向量相似度的高效检索。
- 检索增强生成 (RAG) 通过在生成答案前，从你的知识库中提取相关信息，显著提升模型输出的精确度。该机制使回答得以锚定于真实文档，而非仅依赖模型训练时的静态数据。
- 模块化 (可轻松替换底层实现)，可组合性 (通过 LangChain 表达式语言[LCEL]将组件串联成工作流，可扩展性 (支持自定义加载器、检索器或工具的接入)。
- 文档（Document）是 LangChain 的基本构建单元，用于封装文本内容及其关联元数据。文档加载器（Document Loaders）从多种数据源（如 PDF文件、网页、数据库）中提取内容，并统一转换为标准化的 `Document` 对象。
- LangChain 的核心组件包括基础文档加载器（`BaseLoader`）（将原始内容提取为文档对象)，文档分割器（`TextSplitters`）（将大

段文本切分为适配模型处理的语义块)，嵌入（`Embedding`）模型(将文本转化为语义向量)，向量数据库（`VectorStores`）(存储与检索嵌入向量)，基础文档检索器（`BaseRetriever`）(查询后端，执行向量查询)，基础大语言模型（`BaseLanguageModel`）(提供统一的大语言模型调用接口)。

- `Runnable`接口通过 LangChain 表达式语言（LCEL）实现所有组件的无缝协同，开发者可使用管道操作符 （`|`） 将多个组件串联，构建复杂、可复用的流水线工作流。
- 提示词工程通过精心设计输入内容，帮助模型准确理解用户意图；少样本提示（Few-shot prompting）则在提示词中嵌入示例，以引导模型学习并复现期望的输出模式。
- 在选择模型时，需平衡三个核心维度：精确度 (更大的模型通常在更大规模的数据集上训练而成)，速度 (更小的模型通常响应更快)，成本 (更大的模型每个token费用更高)．客户服务聊天机器人倾向于优先考虑速度与成本，而法律分析工具则更重视准确性。
- 在同一工作流中，可为不同任务选用差异化模型：例如，使用GPT-5-nano实现快速摘要生成，GPT-5-mini负责答案综合，而完整版 GPT-5 用于路由复杂查询。这种分层策略能系统性优化整体的准确性/速度/成本之间的平衡。
- 在检索增强生成（RAG）与微调（fine-tuning）之间做选择时：检索增强生成(RAG) 在运行时动态注入外部知识（降低成本和重新训练成本的需求）；而微调则将领域专业知识直接嵌入模型，提升特定任务的推理效率与一致性。

2 以编程方式执行提示词

本章涵盖以下内容

- 提示词与提示词工程
- 不同类型的提示词及其结构
- 利用单样本、双样本或少样本学习来提升提示词响应效果
- 在ChatGPT和OpenAI API中使用提示词的示例

人工智能应用主要通过*提示词*与大语言模型进行交互—这些结构化的输入旨在引导模型的行为。这有点像给一位才华横溢但经验不足的同事下达指令：你的指示越清晰、越具体，得到的结果就越好。为了获得准确且相关的输出，提示词需要根据任务精心设计与定制。在实战中，提示词设计是影响应用性能表现的最重要因素之一。

提示词工程—即设计和优化提示词以引导大语言模型输出的实践—是构建大语言模型应用的核心技能。你将花费大量时间创建、测试并迭代提示词，以确保你的系统能够提供可靠、高质量的结果。

在本章中，您将从提示词设计的基础知识开始，逐步过渡到更复杂的技术，例如思维链（CoT）。LangChain的提示词工程工具套件，包括`PromptTemplate`和`FewShotPromptTemplate`，这将是您学习如何在应用程序中充分利用大语言模型全部潜力的关键资源。

2.1 以编程方式运行提示词

大语言模型应用依赖精心设计的提示词来生成补全内容，这些内容随后会传递给链中的下一个组件。与在ChatGPT等界面中手动输入的提示词不同，LangChain的提示词通常作为更大工作流的一部分，通过编程方式构建并发送给大语言模型。以下章节首先介绍如何直接使用

OpenAI API来设置和执行提示词，然后我们将探讨如何在LangChain内部运行和管理提示词。在深入之前，请确保你已经掌握了这些基础知识：

- 你拥有一个OpenAI API key(密钥)。
- 你知道如何创建一个Python Jupyter Notebook环境。

如果您不熟悉这些任务，请查阅附录A获取指导，并按照附录B中的说明设置Jupyter Notebook环境。

2.1.1 本章环境设置

假设您已完成所有先决条件，本节将指导您设置用于提示词工程的Jupyter Notebook环境，本章所有示例都将使用此环境。在本章及本书后续部分中，我将演示如何在Windows系统上设置Jupyter。如果您使用其他操作系统，请参阅附录B获取针对特定平台的安装说明。

打开您操作系统的终端（此处以Windows命令提示符为例），创建一个名为`ch02`的新项目文件夹。或者，您也可以克隆本书关联的GitHub仓库：https://mng.bz/YZma。然后，进入`ch02`文件夹：

```
C:\Github\building-llm-applications\ch02>
```

使用`venv`创建并激活虚拟环境：

```
C:\Github\building-llm-applications\ch02>python -m venv env_c
h02
C:\Github\building-llm-applications\ch02>.\env_ch02\Scripts\a
ctivate
```

现在你应该能看到更新后的操作系统提示符，在前面显示环境名称`(env_ch02)`：

```
(env_ch02) C:\Github\building-llm-applications\ch02>
```

激活虚拟环境之后，您现在已经为实现一个用于通过OpenAI模型执行提示词的Jupyter Notebook做好了准备。如果您从GitHub克隆了代码库 (https://mng.bz/YZma)或从曼宁出版社网站下载了zip格式的代码文件，您可以按如下方式安装Jupyter和OpenAI包：

```
(env_ch02) C:\Github\building-llm-applications\ch02>
↪pip install -r requirements.txt
```

然后，您可以启动notebook：

```
(env_ch02) C:\Github\building-llm-applications\ch02>
↪jupyter notebook 02-prompt_examples.ipynb
```

如果你决定从头开始本地构建所有内容，请确保安装了`requirements.txt`中列举的相同版本的软件包—这将有助于你避免后续的版本冲突。在本书的剩余部分，我将假设你已从GitHub克隆了项目仓库或从曼宁出版社网站下载了源代码。

大概1分钟后，notebook和OpenAI包应该就安装完成了。你可以通过执行以下命令来启动Jupyter Notebook：

```
(env_ch02) C:\Github\building-llm-applications\ch02>jupyter n
otebook
```

几秒钟后，在终端中您应该会看到一些输出。随后，浏览器窗口将会打开`http://localhost:8888/tree`地址。通过选择文件 > 新建 > Notebook来创建Notebook，如图2.1所示，然后重命名文件`prompt_examples.ipynb`。

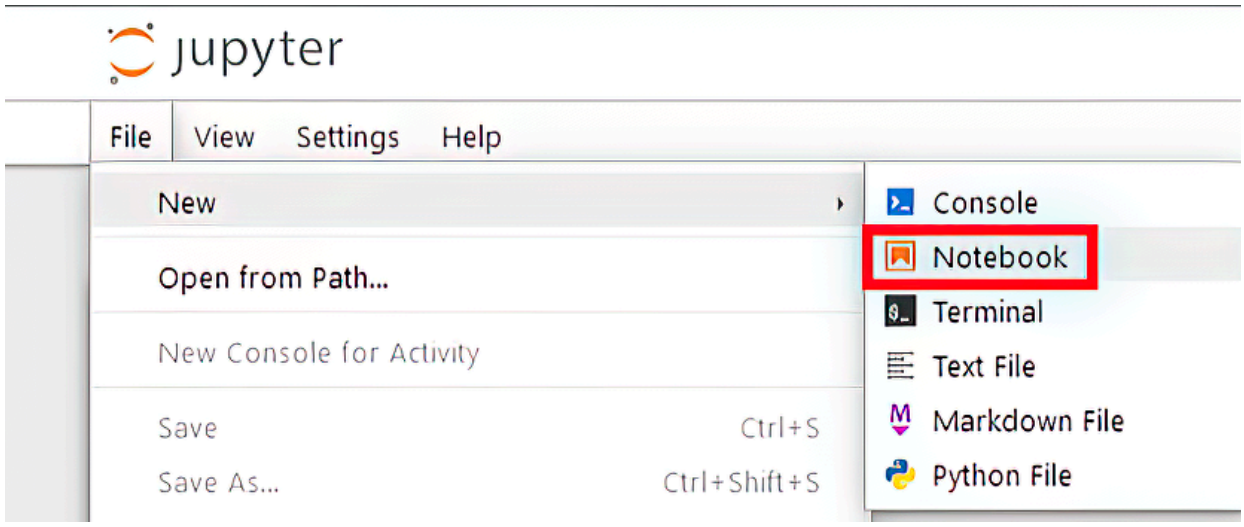

图 2.1 创建一个新的Jupyter Notebook

创建notebook后，前往Jupyter菜单，选择文件 > 重命名，将notebook文件命名为`02-prompt_examples.ipynb`。现在您已准备好在notebook单元格中输入代码（如果你是从GitHub获取的notebook，直接执行即可）。

> **提示** 如果您不熟悉Jupyter Notebook，请记住按Shift-Enter来执行每个单元格中的代码。

在本章的后续部分，我假设您已按照本节所述，设置好了虚拟环境、安装好了OpenAI库并启动了Jupyter Notebook实例。

2.1.2 极简提示词执行

在notebook的第一个单元格中，导入所需的库，并以安全的方式获取OpenAI API密钥（切勿硬编码密钥，以防被滥用）：

```
from openai import OpenAI
import getpass

OPENAI_API_KEY = getpass.getpass('Enter your OPENAI_API_KEY')
```

输入您的OpenAI API密钥后（只需按下Enter键，无需Shift键），按如下方式设置OpenAI客户端：

```
client = OpenAI(api_key=OPENAI_API_KEY)
```

在下一个notebook单元格中，输入并执行以下代码：

```
prompt_input = """Write a concise message to remind
users to be vigilant about phishing attacks."""
# 上述提示词的中文翻译
prompt_input_ZH_CN = """编写一条简洁的信息，提醒用户警惕网络钓鱼攻
击。"""
response = client.chat.completions.create(
  model="gpt-5-nano",
  messages=[
    {"role": "system", "content": "You are a helpful assistan
t."}, //你是一个有用的助手
    {"role": "user", "content": prompt_input}
  ]
)

print(response)
```

> **注意** 为了维持较低的执行成本，我们使用GPT-5-nano模型，这是GPT-5系列中最小且最经济的模型。您将在本书的大部分内容中看到它的使用。不过，如果您希望获得更高的精确度，您可以自由切换到GPT-5-mini或GPT-5。您可以在https://platform.openai.com/docs/models了解更多关于OpenAI模型的详细信息。

输出结果看起来将如下所示（尽管由于大语言模型的不确定性特性，可能会略有不同）：

```
ChatCompletion(id='chatcmpl-CFkN43Xs80ohhDJVIDeRUSID8RXNo', c
hoices=[Choice( finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage( content='Be vigilant against p
hishing: verify the sender, hover over links to check URLs, d
on’t click suspicious attachments or share passwords, and rep
ort anything suspicious to IT.', refusal=None, role='assistan
t', annotations=[], audio=None, function_call=None, tool_call
s=None))], created=1757869206, model ='gpt-5-nano-2025-08-0
7', object='chat.completion', service_tier='default', system_
fingerprint=None, usage=CompletionUsage(completion_tokens=55
3, prompt_tokens=32, total_tokens=585, completion_tokens_deta
ils= CompletionTokensDetails(accepted_prediction_tokens=0, au
dio_tokens=0, reasoning_tokens=512, rejected_prediction_token
s=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=
0, cached_tokens=0)))
```

为了获得更清晰的输出，请探索响应对象的属性和特性：

```
print(response.choices[0].message.content)
```

您将看到类似如下的输出：

```
      Be vigilant against phishing: verify the sender, hover
over links to check URLs, don’t click suspicious attachments
or share passwords, and report anything suspicious to IT.

      译文：警惕网络钓鱼：核实发件人身份，鼠标悬停于链接之上以检查访问网
址，不要点击可疑附件或分享密码，发现可疑情况请及时向IT部门报告。
```

> **注意** 如果您不熟悉以上Python语法，其实我用来构建提示词的三对双引号 (`"""`)允许您以非常易读的方式输入跨多行的文本块，而无需引入换行符(`\n`)，这对于捕获提示词来说是理想的选择。

CHAT.COMPLETIONS.CREATE 函数

`chat.completions.create`函数是 OpenAI Completions API 的一部分，是与 OpenAI 大语言模型交互的主要方式。理解其函数签名至关重要：

```
client.chat.completions.create(
  model="gpt-5-nano",
  messages=[
    {"role": "system", "content": "You are a helpful assist
ant."},
    {"role": "user", "content": prompt_input}
  ]
)
```

以下是参数的解释说明：

- `model`—它指的是您希望使用的OpenAI模型。可选范围从GPT-5（最准确但也最慢且最昂贵）到GPT-5-nano（最快且最便宜，尽管在某些任务上准确性较低）。GPT-5-mini提供了一个稳健的中间选择，在速度、精确度和成本之间取得了平衡。
- `messages`—提示词被表达为一个消息列表，每条消息都有一个角色（例如，系统、用户或助手）和一些包含需要遵循的指令或文本的内容。这种格式遵循了OpenAI构建对话输入的规范。

尽管Responses API现已成为OpenAI的首选接口—尤其适用于基于智能体的工作流—但本书主要使用Completions API，通常通过LangChain中的`ChatOpenAI`包装器间接访问。原因是Completions不仅受到OpenAI模型的支持，还受到许多开源大语言模型的支持，为您提供了更大的灵活性。我们将在第11章深入探讨AI智能体时，重新讨论Responses API。

既然你已经掌握了以编程方式提交提示词的基本方法，那么现在你可以开始学习并使用更复杂的提示词了。

2.2 使用LangChain运行提示词

在转向更高级的提示词之前，让我先展示如何在LangChain中复现相同的示例，以帮助您熟悉其对象模型。在接下来的章节中，我将演示LangChain如何简化更复杂提示词的实现，而这些提示词如果从头开始实现将会更具挑战性。

如果您按照第2.1节的说明安装了所需的Python包，那么您应该已经安装了`langchain`和`langchain-openai`包。现在，您已准备好使用这些库并实例化一个与大语言模型的连接：

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,
                 model_name="gpt-5-nano")
```

现在，您可以按照以下方式实例化并执行之前看到的提示词：

```
prompt_input = """Write a concise message to remind
users to be vigilant about phishing attacks."""
# 上述提示词的中文翻译
prompt_input_ZH_CN = """编写一条简洁的信息，提醒用户警惕网络钓鱼攻
击。"""

response = llm.invoke(prompt_input)
print(response.content)
```

它将返回类似如下的输出：

```
    Stay vigilant against phishing: verify the sender, hover
over links to check URLs before clicking, never share passwor
ds, and report suspicious messages.

    译文：警惕网络钓鱼：核实发件人身份，鼠标悬停于链接之上以检查访问网
址，不要点击可疑附件或分享密码，发现可疑情况请及时向IT部门报告。
```

2.3 提示词模板

在构建大语言模型应用时，创建能够通过参数整合用户输入的灵活提示词模板至关重要。LangChain通过其`PromptTemplate`类简化了这一过程，它允许您管理和复用参数化提示词，而无需编写自定义函数。这不仅简化了动态提示词的创建，还提升了大语言模型交互的效率和适应性，稍后我将进行演示。

2.3.1 使用Python函数实现提示词模板

为了说明模板的概念，让我们创建一个文本摘要模板，它需要请求文本内容、期望的摘要长度以及偏好的语气风格。你可以通过一个简单的Python函数来实现：

```
def generate_text_summary_prompt(text, num_words, tone):
    return f"You are an experienced copywriter. Write a
    ↪{num_words} words summary of the following text, using a
    ↪{tone} tone: {text}"
```

让我们使用提示词模板来生成一个提示词，然后像往常一样通过LangChain的`ChatOpenAI`包装器来执行它：

```
segovia_aqueduct_text = """The Aqueduct of Segovia (Spanish:
Acueducto de Segovia) is a Roman aqueduct in Segovia, Spain.
It was built around the first century AD to channel water fro
m
springs in the mountains 17 kilometres (11 mi) away to the
city's fountains, public baths and private houses, and was in
use until 1973.
Its elevated section, with its complete arcade of 167 arches,
is one of the best-preserved Roman aqueduct bridges and the
foremost symbol of Segovia, as evidenced by its presence on t
he
city's coat of arms.
The Old Town of Segovia and the aqueduct, were declared a UNE
SCO
World Heritage Site in 1985. As the aqueduct lacks a legible
inscription (one was apparently located in the structure's at
tic,
or top portion[citation needed]), the date of construction ca
nnot be
definitively determined. The general date of the Aqueduct's
construction was long a mystery, although it was thought to h
ave
been during the 1st century AD, during the reigns of the Empe
rors
Domitian, Nerva, and Trajan. At the end of the 20th century,
Géza Alföldy deciphered the text on the dedication plaque by
studying the anchors that held the now missing bronze letters
in place. He determined that Emperor Domitian (AD 81–96) orde
red
its construction[1] and the year 98 AD was proposed as the mo
st
likely date of completion.[2] However, in 2016 archeological
evidence was published which points to a slightly later date,
after 112 AD, during the government of Trajan or in the
beginning of the government of emperor Hadrian,
from 117 AD."""

# 上述提示词译文
segovia_aqueduct_text_ZH_CN = """
```

```
塞戈维亚水道（西班牙语：Acueducto de Segovia）是位于西班牙塞戈维亚的一座罗马水道。它建于公元1世纪左右，将17公里（11英里）外山中的泉水引至城内的喷泉、公共浴池和私人住宅，一直使用至1973年。
其高架部分及其完整的167拱拱廊，是保存最完好的罗马水道桥之一，也是塞戈维亚最重要的象征，这一点从它出现在该市的市徽上可见一斑。
塞戈维亚老城区和水道于1985年被联合国教科文组织列为世界遗产。由于水道缺乏清晰的铭文（铭文显然位于建筑物的阁楼或顶部[需提供引用]），因此无法确切确定其建造日期。水道的建造日期长期以来一直是个谜，尽管人们认为它建于公元1世纪，即多米提安、涅尔瓦和图拉真皇帝在位期间。20世纪末，盖佐·阿尔福尔迪通过研究固定现已缺失的青铜字母的锚，破译了铭牌上的文字。他确定是多米提安皇帝（公元81-96年在位）下令建造的[1]，并认为公元98年是最可能的完工日期[2]。然而，2016年公布的考古证据指出，完工日期稍晚，是在公元112年之后，即在图拉真政府时期或公元117年哈德良皇帝政府初期。
"""

input_prompt = generate_text_summary_prompt(
    text=segovia_aqueduct_text,
    num_words=20,
    tone="knowledgeable and engaging")

response = llm.invoke(input_prompt)
print(response.content)
```

您应该会看到类似如下的输出：

```
    The Aqueduct of Segovia, built in the 1st century AD, is a well-preserved Roman structure and a UNESCO World Heritage Site.

    译文：塞戈维亚水道建于公元1世纪，是一座保存完好的罗马建筑，也是联合国教科文组织认定的世界遗产。
```

2.3.2 使用 LangChain 的 PromptTemplate

使用LangChain，您无需手动实现提示词模板函数。相反，您可以使用便捷的`PromptTemplate`类来处理参数化模板。以下是其使用方法：

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template(
"""You are an experienced copywriter.
Write a {num_words} words summary of the following text,
using a {tone} tone: {text}""")
```

要使用提示词模板，请创建一个prompt实例，并使用您的参数进行格式化：

```
prompt = prompt_template.format(
    text=segovia_aqueduct_text,
    num_words=20,
    tone="knowledgeable and engaging")
```

然后，使用格式化后的提示词调用`ChatOpenAI`客户端：

```
response = llm.invoke(prompt)
print(response.content)
```

它将生成如下显示的类似输出（正如你现在所理解的，确切的输出可能与书中打印的不同，因此我不会在下一页重复这一点）：

```
    The Aqueduct of Segovia, a Roman marvel built in the 1st
century AD, channels water to Segovia's fountains and baths.

    译文：塞戈维亚水道，这座建于公元1世纪的罗马奇迹，将水引向塞戈维亚的喷泉和浴场。
```

让我们暂时从LangChain的编码中稍作休息。我想更深入地探讨提示词工程。在接下来的几页中，你会看到，一个好的提示词可以包含多种元素，这取决于你的目标、任务复杂度和期望的准确性。通过学习不同的提示词变体，你将能够处理简单和复杂的提示词，并根据具体情况调整其复杂度。

2.4 提示词类型

在LangChain应用中，创建有效的提示词对于获得最佳结果至关重要。无论您的应用是专注于文本分类、情感分析、摘要生成、文本生成还是问答系统，每项任务都需要精心设计的提示词。在接下来的章节中，我们将深入探讨如何为这些特定任务量身定制提示词，确保您的应用按预期运行。

尽管这些提示词技术对于开发LangChain应用程序至关重要，但为了保持简洁并更清晰地解释提示词工程概念，我们暂时将使用ChatGPT。您在这里学到的模式将直接适用于我们在后续章节中使用LangChain构建的大语言模型应用程序。

2.4.1 文本分类

在分类任务中，目标是将输入文本分配到多个预定义类别中的其中一个。以下提示词展示了这一概念—请尝试将其输入到ChatGPT中，我们将在本节中全程使用它：

指令　将以下文本归类到以下类别之一：历史、科技、园艺。

文本　耳机为音乐爱好者和游戏玩家提供了沉浸式音频体验。

文本“耳机为音乐爱好者和游戏玩家提供了沉浸式音频体验。” 应归类为 “科技”，因为它讨论了与技术相关的产品及其功能。

输出内容过于详细。让我们修改提示词，指定只显示类别。

指令　将以下文本归类到以下类别之一：历史、科技、园艺。

文本　耳机为音乐爱好者和游戏玩家提供了沉浸式音频体验。

输出　只输出分类名称

“科技”

现在通过应用相同的提示词并仅修改文本来对另一个句子进行分类：

指令　将以下文本归类到以下类别之一：历史、科技、园艺。

文本　尤利乌斯·凯撒是一位罗马将军和政治家，他在罗马共和国的覆灭和罗马帝国的崛起中扮演了关键角色。

输出　只输出分类名称

“历史”

总而言之，在一个标准的文本分类提示词中，你会找到三个组成部分：指令、输入文本和输出规范。接下来，让我们深入探讨一个略微专业化的文本分类：情感分析。

2.4.2 情感分析

*情感分析*是一种特定的文本分类类型，旨在判断给定文本被感知为积极、中性还是消极。以下是三个示例提示词（及期望回复），您可以独立进行实验：

指令　将以下文本分类为积极、中性或消极

文本　苹果公司公布了出色的季度业绩

“积极”

指令　将以下文本分类为积极、中性或消极

文本　这是本月我看过的最糟糕的电影

“消极”

指令　将以下文本分类为积极、中性或消极

文本　顾客购买了一条绿色连衣裙

“中性”

如果您有一个文本列表，并希望集中分析它们的情感倾向，可以将它们全部输入到同一个提示词中：

指令　将以下股票报告的情绪分类为积极、中性或消极

股票1: 苹果：新款iPhone的发布取得了成功

股票2: 英伟达：销售额正受到消费者对大语言模型需求的推动

股票3: GX石油：由于可再生能源的增加，碳基能源的需求正在下降

输出　输出一个包含“股票名称”、“情感倾向”列的表格

```
|    股票名称     |    情感倾向     |
|-------------|-------------|
|     苹果       |      积极       |
|    英伟达      |      积极       |
|    GX石油      |      消极       |
```

正如你所见，在这个场景中，输出以表格形式呈现，并且可以轻松地通过编程方式处理。接下来，让我们为最常见的大语言模型应用场景之一—文本摘要—创建一个提示词。

2.4.3 文本摘要

创建*文本摘要*提示非常简单：你只需要请求对输入文本进行摘要生成，并指定你偏好的长度。

> 指令　为下列文本写一段30字的文本摘要
>
> 文本　20世纪80年代的家用个人电脑是个人计算领域的标志性先驱。诸如Apple II、IBM PC和Commodore 64等计算机彻底改变了我们的日常生活和工作方式。它们将文字处理、游戏以及家庭计算的可能性带入了我们的日常生活。正如苹果Macintosh所展示的用户图形界面，改变了我们与机器交互的方式。这些个人电脑，尽管以今天的标准来看性能较弱，却点燃了一场数字革命，它塑造了我们今天所熟知的技术格局。它们在历史上占据着特殊的地位，象征着计算领域新时代的黎明，为现代数字时代奠定了基础。
>
> “20世纪80年代的家用个人电脑，如Apple II、IBM PC、Commodore 64，开创了个人计算的先河，引入了文字处理、游戏和用户图形界面。它们塑造了当今的科技格局，象征着一个数字时代的黎明。”

你也可以要求大语言模型为它已经熟悉的文本提供摘要：

> 指令　请用一句话总结《指环王》
>
> “《指环王》讲述了一群各具特色的角色展开史诗般的旅程，他们试图摧毁一枚强大的戒指，将中土世界从黑暗魔君索伦手中拯救出来。”

2.4.4 文本创作

大语言模型属于人工智能的一个类别，被称为*生成式AI*。虽然大语言模型能够处理音频和视频，但当接收到文本提示词时，它们可以基于一系列事实生成新的内容，例如一个段落或句子：

撰写一篇关于潜水手表的文章，需提及以下事实：

- 劳力士蚝式恒动手表是首批具备防水性能的手表之一。
- 欧米茄于1948年推出海马系列，成为首款专为潜水设计的手表。
- 1955年，海马系列创下62.5米潜水深度的世界纪录。
- 劳力士于1953年推出首款潜航者型手表。
- 潜航者型手表因与詹姆斯·邦德的关联而广受欢迎，直至1995年。
- 自1995年起，詹姆斯·邦德开始佩戴欧米茄海马系列手表。

“劳力士蚝式恒动手表于20世纪初问世，在潜水手表的发展历程中发挥了开创性作用。其卓越的防水设计成为关键突破，为真正意义上的防水手表奠定了技术基础，这一创新显著提升了手表在水下极端环境中的可靠性与实用性。

欧米茄作为备受尊敬的瑞士制表品牌，于1948年推出海马系列，彻底改变了潜水手表领域。这款手表专为潜水员设计，将优雅风格与实用功能完美融合。海马系列不仅在设计上突破创新，更在深海环境中验证了其卓越性能—1955年，它创下潜水深度62.5米的世界纪录，成为当时水下耐压能力的标杆。

不甘示弱，劳力士于1953年推出了首款潜航者型手表，加入了这场竞争。这款标志性手表迅速成为水下探索与冒险的代名词。其与虚构的英国间谍詹姆斯·邦德的关联，进一步推动了它的成功。然而，流行趋势在1995年发生了转变，当时由皮尔斯·布鲁斯南饰演的詹姆斯·邦德在《黄金眼》中大胆地佩戴上了欧米茄海马手表。这标志着海马腕表新时代的开始，因为它成为了世界上最著名特工的首选手表。”

您可能会注意到，ChatGPT利用有限的信息创作了一篇相当长的文章。另一方面，您可能期望一篇更短的文章，并且可能不喜欢它使用复杂词汇或略显正式的语气。在这种情况下，请尝试更系统地重构提示词，以使生成的文本符合您偏好的长度、词汇和语气：

角色设定　你是一位手表爱好者，经验丰富，擅长为面向大众的手表杂志撰写高质量文章。

指令　撰写一篇关于潜水手表的文章

需要提及的事实：

- 劳力士蚝式恒动手表是首批具备防水性能的手表之一。
- 欧米茄于1948年推出海马系列，成为首款专为潜水设计的手表。
- 1955年，海马系列创下62.5米潜水深度的世界纪录。
- 劳力士于1953年推出首款潜航者型手表。
- 潜航者型手表因与詹姆斯·邦德的关联而广受欢迎，直至1995年。
- 自1995年起，詹姆斯·邦德开始佩戴欧米茄海马系列手表。

语气　采用一种庄重、博学但又不过于正式的语气，并使用日常词汇。

输出　文章长度最多应最多控制在150个字以内

“**潜水手表: 探险精神与时尚风范并存的手表杰作**

潜水手表长久以来因其功能与美学的完美融合而备受推崇，既是冒险者的理想伴侣，也是时尚品味者的不二之选。

劳力士凭借蚝式恒动手表率先开创了防水技术，为手表制造开启了一个全新的时代。

然而，欧米茄于1948年以海马系列掀起革命，成为全球首款专为潜水员设计的手表。它不仅在设计上兼顾优雅与实用，更在深海中验证了其卓越性能—1955年，它创下62.5米潜水深度的世界纪录，彰显了其在极端环境下的可靠表现。

劳力士并未落后，于1953年推出潜航者型手表，这款手表因与银幕上詹姆斯·邦德角色的紧密关联，迅速成为探险精神的代名词。

说到詹姆斯·邦德，值得一提的是，1995年这位传奇特工开始佩戴欧米茄海马系列，这一转变进一步巩固了海马系列作为兼具精致气质与坚韧性能的手表地位，专为那些敢于探索陆地与海洋之人

而生。这些潜水手表早已超越计时工具的范畴，成为冒险精神与优雅风格的永恒象征。”

通过向提示中添加更多细节，文章已完全按照您的要求生成。

注意　创建一个结构清晰、包含明确定义部分（角色、指令、事实、语气、输出格式）的提示词，有助于大语言模型更好地理解您的要求，并生成预期的文本。

2.4.5 问答

大语言模型“理解”文本的能力使其不仅适用于摘要生成，也适用于检索特定信息，这被称为*问答*使用场景。现在，让我们回到摘要示例中使用的同一段文本，并针对其内容提出一些具体问题。

指令　阅读以下文本，并告诉我80年代哪些家用个人电脑比较流行。

文本　20世纪80年代的家用个人电脑是个人计算领域的标志性先驱。诸如Apple II、IBM PC和Commodore 64等计算机彻底改变了我们的日常生活和工作方式。它们将文字处理、游戏以及家庭计算的可能性带入了我们的日常生活。正如苹果Macintosh所展示的用户图形界面，改变了我们与机器交互的方式。这些个人电脑，尽管以今天的标准来看性能较弱，却点燃了一场数字革命，它塑造了我们今天所熟知的技术格局。它们在历史上占据着特殊的地位，象征着计算领域新时代的黎明，为现代数字时代奠定了基础。

输出　仅以列表形式输出电脑的名称

20世纪80年代流行的家用个人电脑有:

1. Apple II
2. IBM PC
3. Commodore 64
4. Apple Macintosh

问答是一种常见的应用场景，因此大多数大语言模型都将Q和A理解为问题和答案的缩写。例如：

> 文本　Java是一种流行的编程语言，它将代码编译成字节码，由Java虚拟机（JVM）执行，从而实现跨平台的应用程序开发。
>
> Q: Java代码在哪里被执行?
>
> A: Java代码由Java虚拟机（JVM）执行。

大语言模型不仅擅长在文本中查找所需信息，还擅长进行逻辑推理。这一点将在下一节中讲解。

2.4.6 推理

虽然推理是一项强大的能力，但大语言模型在这方面仍在不断改进。因此，你可能需要更高级的提示词工程技术来获得想要的结果。以下是一个涉及数学推理的简单示例：

> 指令　计算这串数字中平方数的和：19, 13, 1, 17, 4, 64, 900
>
> “这串数字19, 13, 1, 17, 4, 64, 900中是平方数的有：1, 4, 64, 900，它们的和为：”
>
> 1 + 4 + 64 + 900 = 969
>
> 因此，这串数字中平方数的和为969。

ChatGPT识别出平方数，将它们挑选出来，然后计算它们的总和。下一个例子更具挑战性，涉及到回文数—那些正读反读都相同的数字，例如1661或27872：

> 指令　求这串数字中回文数的和：13, 1331, 121, 73, 99, 56, 232, 7

我得到了如下输出：

“这串数字13, 1331, 121, 73, 99, 56, 232, 7中是回文数的有：1331, 121, 232, 7，它们的和为：”

1331 + 121 + 232 + 7 = 1691

因此，这串数字中回文数的和为1691。

这是错误的！回文数是指正读和反读都相同的数字。因此，99本应被视为回文数，但它却没有被纳入计算。

坦白地说：这个输出是我在撰写本书早期阶段遇到的。不过，你现在应该会看到一个不同的结果，更重要的是，你将会得到一个准确的输出。OpenAI在后续模型版本中解决这个问题之前，解释以下我是如何纠正ChatGPT的这个错误的，仍然很具有价值。

那么我是如何解决这个问题的呢？由于早期的大语言模型版本未能完全理解回文的概念，我“教会”了ChatGPT正确识别所有相关案例，具体方法将在下一节中阐述。在继续之前，请花点时间回顾一下表2.1中总结的您已学习的提示词类型。

表格 2.1 提示词类型与提示词示例

提示词类型	提示词示例
文本分类	将以下文本归类到以下类别之一：历史、科技、园艺。
情感分析	将以下文本分类为积极、中性或消极
文本摘要	为下列文本写一段30字的文本摘要
文本创作	撰写一篇关于潜水手表的文章，需提及以下事实： • 劳力士蚝式恒动手表是首批具备防水性能的手表之一。 • 欧米茄于1948年推出海马系列，成为首款专为潜水设计的手表。
问答	阅读以下文本，并告诉我80年代哪些家用个人电脑比较流行。
推理	计算这串数字中平方数的和：19, 13, 1, 17, 4, 64, 900

2.5 详细推理

在开发大语言模型应用时，特别是那些需要复杂推理的应用，理解不同提示词技术的局限性至关重要。例如，*零样本学习*是一种不提供任何示例的方法，期望大语言模型仅根据给定的指令来理解和执行任务。虽然这种方法在某些场景下效果良好，但对于更复杂的任务（例如教大语言模型识别回文），它并不能总产生准确的结果。在接下来的章节中，我将演示如何超越*零样本学习*，优化大语言模型提示词，以在推理任务中获得更可靠的性能。

2.5.1 单样本学习

当你在提示词中只提供一个示例时，这被称为*单样本学习*。以下是我通过提供单个示例来尝试解决回文问题的方法：

> 指令　求这串数字中回文数的和：13, 1331, 121, 73, 99, 56, 232, 7
>
> 示例　33是一个回文数
>
> “我为之前的疏忽致歉。让我们重新正确识别回文数并计算这串数字中的回文数之和：13, 1331, 121, 73, 99, 56, 232, 7。”
>
> “这串数字中是回文数的有：1331, 121, 232, 7，它们的和为：”
>
> 1331 + 121 + 232 + 7 = 1691
>
> 因此，这串数字中回文数的和确实为1691。

遗憾的是，即使使用了单样本学习，问题依然存在。因此我调整了提示词并加入了两个示例。

2.5.2 双样本学习

以下是我使用的修订后的提示词，以及它生成的输出。这个输出展示了模型如何理解修订后的指令，并在这个双样本学习示例的上下文中应用它们。

> 指令　求这串数字中回文数的和：13, 1331, 121, 73, 99, 56, 232, 7
>
> 示例:
>
> 33是一个回文数
>
> 44是一个回文数
>
> “当然，让我们找出这串数字：13, 1331, 121, 73, 99, 56, 232, 7中的所有回文数，并计算它们的和。”
>
> “这串数字中是回文数的有：1331, 121, 232, 7，它们的和为：”
>
> 1331 + 121 + 232 + 7 = 1691
>
> 因此，这串数字中回文数的和为1691。

依然不正确：99这个数字再次被忽略了。在又一次尝试失败后，我探索了不同的方法，并找到了一个标准解决方案。它建议将问题分解为多个步骤，并为每个步骤提供清晰的说明。这类似于指导一位可能在某个具体步骤上遇到困难的同事。

2.5.3 提供步骤

这是我输入的修改后的提示词，以及它生成的相应输出。此输出展示了模型如何遵循修订后的指令，并应用了此示例所需的逐步推理方法。

> 指令　求这串数字中回文数的和：13, 1331, 121, 73, 99, 56, 232, 7
>
> 示例:
>
> 33是一个回文数
>
> 44是一个回文数
>
> 步骤: 1) 识别回文数; 2) 对回文数求和
>
> “当然，让我们按照您的指示逐步进行”:
>
> 第1步: 识别这串数字中的回文数
>
> 这串数字中的回文数有：
>
> - 1331
> - 121
> - 99
> - 232
> - 7
>
> 第2步: 对它们求和
>
> 1331 + 121 + 99 + 232 + 7 = 1790
>
> 因此，这串数字中回文数的和为1790。

终于，我搞定了这个问题！有趣的是，ChatGPT似乎也经历了类似的训练过程（当然规模要大得多）来解决这个问题。为了说明这一点，如果你输入原始提示词：

> 指令　求这串数字中回文数的和：13, 1331, 121, 73, 99, 56, 232, 7

现在你将看到如下输出：

“让我们找出给定数字中的回文数：”

- 1331是一个回文数
- 121是一个回文数
- 99是一个回文数
- 232是一个回文数
- 7是一个回文数

现在，让我们将这些回文数相加：

1331 + 121 + 99 + 232 + 7 = 1790

因此，这串数字中回文数的和为1790。

看起来ChatGPT正在像我一样逐步解决问题。像GPT-5 Thinking这样的推理模型通常会自行分解问题，而无需被告知要遵循哪些步骤。然而，在更复杂的情况下—尤其是当步骤是确定的时候—在提示词中明确列出它们仍然很有用。这确保了大语言模型遵循预期的顺序，而不是自行发明步骤，后者可能会导致错误。

我觉得分享一个已不再适用的例子很有意思。这是你在使用大语言模型时可能会遇到的情况：你可能会花费时间，利用高级提示词工程技术来教导模型执行某些任务，结果却发现新版本的模型无需任何额外训练就能处理这些任务。例如，OpenAI的GPT-5模型仅需极简提示词就能执行推理任务。而这些任务在过去则需要更复杂的提示词技巧，比如你即将要学习的少样本提示词技术。

2.5.4 少样本学习

在某些情况下，仅提供两个示例可能不足以教会大语言模型理解一个新概念。此时，你需要提供多个示例，这被称为*少样本学习*。为了演示这一点，让我来设计一个游戏，在这个游戏中，一个数字可以被归类为Abra、Kadabra或Abra Kadabra。让我们看看在没有提供任何分类规则信息时会发生什么。

> 将下列数字归类为Abra、Kadabra或Abra Kadabra：
>
> 3, 5, 8, 2, 7, 12, 13

> “将数字分类为Abra、Kadabra或Abra Kadabra，并不遵循传统的数学模式。这些分类只是词语，与数字无关，因此无法依据任何数学或逻辑标准，将你提供的数字归类为Abra、Kadabra或Abra Kadabra。”

ChatGPT并未出现任何幻觉，反而恰当地提出了您尚未阐明的分类规则。现在我将演示一个提示词，它并非直接解释规则，而是基于一个特定的数字列表提供分类规则解读，这个列表与我最初要求处理的数字不同：

> 将下列数字归类为Abra、Kadabra或Abra Kadabra：
>
> 3, 4, 5, 7, 8, 10, 11, 13, 35
>
> 示例:
>
> 6 // 不能被5整除，不能被7整除 // 什么都不是
>
> 15 // 能被5整除，但不能被7整除 // Abra
>
> 12 // 不能被5整除且不能被7整除 // 什么都不是
>
> 21 // 不能被5整除，但能被7整除 // Kadabra
>
> 70 // 能被5整除，且能被7整除 // Abra Kadabra

请注意我在每个示例中采用的格式：我使用双斜杠（//）来分隔每个字段，尽管我没有明确说明每个字段的含义。我依赖大语言模型来解读它。以下是输出结果：

> “让我们根据数字是否能被5和/或7整除，将其归类为Abra、Kadabra或Abra Kadabra：”
>
> 3: 不能被5整除且不能被7整除 - 什么都不是

4: 不能被5整除且不能被7整除 - 什么都不是

5: 能被5整除，但不能被7整除 - Abra

7: 不能被5整除，但能被7整除 - Kadabra

8: 不能被5整除且不能被7整除 - 什么都不是

10: 能被5整除，但不能被7整除 - Abra

11: 不能被5整除且不能被7整除 - 什么都不是

13: 不能被5整除且不能被7整除 - 什么都不是

35: 能被5整除，且能被7整除 - Abra Kadabra

那么，分类如下：

Abra: 5, 10, 35

Kadabra: 7

Abra Kadabra: 35

什么都不是: 3, 4, 8, 11, 13

ChatGPT不仅提供了准确的结果，还解释了其背后的推理过程。它成功地从给定的示例中推导出通用规则，并清晰地阐述了这些规则。这确实令人印象深刻，不是吗？

注意 您可能熟悉一个类似的数字分类游戏或"可除性测试算法"，正如它在 Stack Overflow (https://mng.bz/GwzJ) 上所称的那样，其中能被3整除的数字被标记为"Foo"，能被5整除的数字被标记为"Bar"，而同时能被3和5整除的数字则被标记为"Foo Bar"。我没有使用这个作为例子，是因为 ChatGPT 已经知道它了，并且可以在没有示例的情况下，使用零样本学习正确地对数字进行分类。

2.5.5 使用LangChain实现少样本学习

让我们使用LangChain来实现AbraKadabra游戏。首先，像之前一样建立与大语言模型的连接：

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,
                 model_name="gpt-5-mini")
```

现在定义并运行你之前在ChatGPT上直接执行的相同少样本提示词：

```
prompt_input = """将下列数字归类为Abra、Kadabra
↪或Abra Kadabra:

3, 4, 5, 7, 8, 10, 11, 13, 35

示例:
6 // 不能被5整除且不能被7整除 // 什么都不是
15 // 能被5整除，但不能被7整除 // Abra
12 // 不能被5整除且不能被7整除 // 什么都不是
21 // 不能被5整除，但能被7整除 // Kadabra
70 // 能被5整除且能被7整除 // Abra Kadabra
"""

response = llm.invoke(prompt_input)
print(response.content)
```

The output is as expected:

```
3 // 不能被5整除且不能被7整除 // 什么都不是
4 // 不能被5整除且不能被7整除 // 什么都不是
5 // 能被5整除，但不能被7整除 // Abra
7 // 不能被5整除，但能被7整除 // Kadabra
8 // 不能被5整除且不能被7整除 // 什么都不是
10 // 能被5整除，但不能被7整除 // Abra
11 // 不能被5整除且不能被7整除 // 什么都不是
13 // 不能被5整除且不能被7整除 // 什么都不是
35 // 能被5整除且能被7整除 // Abra Kadabra
```

输出结果是正确的，但实现方式并不理想，因为示例被硬编码在提示词中。LangChain 提供了一种更简洁的创建少样本提示词的解决方案。它允许你将训练示例与提示词模板分离，并在稍后注入它们，如下面的代码清单所示。

清单 2.1 使用 FewShotPromptTemplate 的少样本提示词

```
from langchain_core.prompts.few_shot import FewShotPromptTemp
late
from langchain_core.prompts.prompt import PromptTemplate
examples = [
  {
      "number": 6,
      "reasoning": "not divisible by 5 nor by 7",
      "result": "None"
  },
  {
      "number": 15,
      "reasoning": "divisible by 5 but not by 7",
      "result": "Abra"
  },
  {
      "number": 12,
      "reasoning": "not divisible by 5 nor by 7",
      "result": "None"
  },
  {
      "number": 21,
      "reasoning": "divisible by 7 but not by 5",
      "result": "Kadabra"
  },
  {
      "number": 70,
      "reasoning": "divisible by 5 and by 7",
      "result": "Abra Kadabra"
  } ]

example_prompt = PromptTemplate(input_variables=["number",
↪"reasoning", "result"],
↪template="{number} \\ {reasoning} \\ {result}")
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    suffix="将下列数字归类为Abra、Kadabra
```

```
    ↪或Abra Kadabra: {comma_delimited_input_numbers}",
    input_variables=["comma_delimited_input_numbers"]
)

prompt_input = few_shot_prompt.format(
↪comma_delimited_input_numbers="3, 4, 5, 7, 8, 10, 11, 13, 3
5.")

response = llm.invoke(prompt_input)
print(response.content)
```

输出结果依然是正确的：

```
3 \ 不能被5整除且不能被7整除 \ 什么都不是
4 \ 不能被5整除且不能被7整除 \ 什么都不是
5 \ 能被5整除，但不能被7整除 \ Abra
7 \ 能被5整除且能被7整除 \ Abra Kadabra
8 \ 不能被5整除且不能被7整除 \ 什么都不是
10 \ 能被5整除，但不能被7整除 \ Abra
11 \ 不能被5整除且不能被7整除 \ 什么都不是
13 \ 不能被5整除且不能被7整除 \ 什么都不是
35 \ 能被5整除且能被7整除 \ Abra Kadabra
```

2.5.6 思维链

最后，我想向您介绍思维链技术，它结合了提供逻辑步骤和少量示例学习。在这里，我虚构了一个新的数学定义：如果一个数列包含至少两个奇数，并且所有奇数的和能被3整除，则该数列被称为"奇异"数列。首先，我检查大语言模型是否不熟悉奇异数列的规则：

> 下面这个数列是否为"奇异"数列？
>
> 3, 4, 5, 7, 10, 18, 22, 24

> "您提供的这个数列看起来是递增的，但它并不遵循常见的数学规律，比如等差数列或等比数列。它是否被视为"奇异数列"取决于您用来定义"奇异数列"的上下文或标准。从数学意义上讲，它可能被

认为是一个不寻常或非标准的数列。如果您有定义“奇异”数列的具体标准或规则，请提供更多细节以便进行更精确的分类。”

正如预想的那样，ChatGPT似乎并不能理解“奇异”数列的概念，这再次证明，值得庆幸的是，它尚未发展出读心术，至少目前如此。

我将基于Wei等人提出的思维链技术（参见 https://arxiv.org/abs/2201.11903），构建一个提示词模板。此提示词是对少量样本学习提示词的扩展，将包含以下内容：

- 若干用于少量样本学习的示例数列
- 每个示例数列中关于判断数列是否为“奇异数列”的逻辑步骤说明

Q: 下面这个数列是否为“奇异”数列?

3, 4, 5, 7, 10, 18, 22, 24

示例:

Q: 这个数列是否为“奇异”数列: 1, 4, 6, 8, 20

A: 1是一个奇数; 我需要至少2个奇数 // 非奇异数列

Q: 这个数列是否为“奇异”数列: 5, 6, 7, 8, 20

A: 5和7是奇数; 5和7的和为12; 12能被3整除 // 奇异数列

Q: 这个数列是否为“奇异”数列: 1, 5, 6, 7, 8, 20

A: 1, 5, 7是奇数; 1, 5, 7的和为13; 13不能被3整除 // 非奇异数列

Q: 这个数列是否为“奇异”数列: 5, 6, 7, 8, 9, 20

A: 5, 7, 9是奇数; 5, 7, 9的和为21; 21能被3整除 // 奇异数列

“感谢您提供的示例和说明。根据您提出的标准：”

数列“3, 4, 5, 7, 10, 18, 22, 24”中有3个奇数: 3, 5, 7。这些奇数之和为15. 15能被3整除. 因此，根据上述标准, 这个数列是**奇异**数列。

ChatGPT给出了正确答案并解释了其思考过程。为了加深你的理解，我尝试使用一组不同的数字进行类似的提问。此外，观察当数列中只有一个奇数时，ChatGPT会如何回应。

在本节中，您已经了解了如何通过提供各种示例或概述必要步骤，向ChatGPT介绍一项新任务。这种方法被称为上下文学习，因为您是在利用提示词的上下文中所提供的信息来指导大语言模型。

定义 *上下文学习* 指的是任何通过提示词上下文中的示例，指导大语言模型完成用户请求的陌生任务或指令的方法。常见的技术包括单样本学习、双样本学习或少样本学习，以及提供逐步指导（通常称为思维链）。与微调相比—微调涉及通过在特定领域的文本上进行训练，基于通用模型创建专用的大语言模型—上下文学习是一种更具成本效益且资源密集度更低的方法。它不需要高端硬件（如GPU），也不需要深入理解Transformer架构。

本节涵盖了多种上下文学习提示词，这些内容总结于表格2.2中。

表格 2.2 上下文学习提示词

上下文学习技术	解释说明
零样本学习	提示词中未提供任何示例。
单样本学习	提示词中只提供了一个示例。
双样本学习	提示词中提供了两个示例。
少样本学习	提示词中提供了若干示例。
思维链(CoT)	提示词中提供了若干示例，并且对于每个示例，实现目标的全部逻辑步骤都给出了清晰的解释。

超越思维链

思维链（CoT）通过将问题分解为更小的逻辑步骤，改进了大语言模型处理复杂推理的方式。然而，它存在一些局限性，例如缺乏更深入的探索或在处理混乱上下文时会遇到困难。两种先进技术—思维树（ToT）和思维线（ThoT）—可以解决这些不足：

- *思维树(ToT)*—ToT通过允许大语言模型探索多种推理路径来提升问题解决能力。传统模型进行逐个token的决策，这限制了它们提前规划或重新审视先前选择的能力。ToT将这一过程结构化为连贯的步骤，使模型能够在推进过程中评估和完善选项（参见https://arxiv.org/abs/2305.10601）。这种方法在需要战略性思考的任务中非常有效，例如24点游戏、创意写作和迷你填字游戏。例如，ToT帮助GPT-4解决了74%的24点游戏问题，而使用CoT仅解决了4%。这种方法有助于提升审慎决策、回溯和战略预见能力。
- *思维线索(ThoT)*—ThoT是一种针对混乱上下文（参见https://arxiv.org/abs/2311.08734）中挑战的解决方案。在混乱上下文中，无关细节会干扰模型或导致错误。受人类认知过程启发，ThoT 通过系统性地分解和分析大段信息，同时聚焦相关细节，有效应对这一问题。该策略可作为模块化插件，与多种模型及提示词技术无缝集成。在 PopQA、EntityQ 以及多轮对话响应等数据集上的实验表明，ThoT 能显著提升推理准确性，其优势在于有效过滤噪声并在复杂上下文中保持注意力集中。

这两种技术都推动着大语言模型超越了简单的响应生成，在结构化和混乱的场景中提升了推理、规划和决策能力。有关技术细节和示例，请参阅已发表的研究论文和代码仓库。

在本章结束之前，让我们尝试从迄今为止分析的各类用例中，提炼出一种标准的提示词格式。

2.6 提示词结构

结合前面各章节的所有提示词要素，可得出以下通用结构：

- *角色设定*—明确指定你希望大语言模型（LLM）扮演的角色。无论是经验丰富的市场营销经理、获奖的科幻作家，还是专业计算机杂志的文稿编辑，都应设定高标准的期望。
- *上下文*—提供详细的上下文信息，帮助大语言模型理解你请求所处的上下文环境。精确的描述有助于提高结果与期望的契合度。
- *指令*—清晰定义你希望大语言模型对输入文本执行的具体操作。无论是翻译、写作、改写、优化、分类、排序还是查找，都需明确说明。
- *输入*—指的是具体的上下文细节，可以是一段文字、一个问题或一系列关键点。你可以选择相应的标签进行标注，例如“文本”、“问题”等等。
- *步骤*—概述大语言模型为生成输出应遵循的处理步骤。
- *语气*—指定大语言模型回答时您期望的语气，如正式、非正式、风趣、热情、冷静、友好等，也可组合使用。
- *输出格式*大语言模型默认以文本形式回答。但你也可以要求以表格、CSV 文件（含列名）、JSON（含属性名）或其他结构化格式呈现。
- *示例*—在大语言模型可能缺乏特定任务（尤其是文本分类和推理任务）足够训练的情况下，可通过示例提供小规模内联训练。即使少量示例也能显著提升回答质量。

此列表可作为组织提示词的指南或检查清单。建议根据实际需求自行调整，选取相关部分并进行个性化修改。以下是一个融合了上述诸多要素、结构清晰的提示词示例：

> 角色设定　您是一位经验丰富的大语言模型（LLM）开发者和知名演讲者。

上下文　你受邀为一场大语言模型活动发表主题演讲。

指令　为演讲撰写点睛之笔。

输入　请包含以下事实：

- 大语言模型(LLM)自2022年11月ChatGPT发布以来已成为业界主流。
- 此后相继推出了LLaMA-2、Falcon180B、Bard等众多广受欢迎的大语言模型及基于LLM的聊天机器人。
- 大语言模型(LLM)的普及程度正逐步与搜索引擎比肩。
- 众多企业正积极将其自身应用与大语言模型进行集成。

语气　采用机智诙谐但引人入胜的语气。

输出格式　将文本分以每段五行,总共两段的格式呈现。

“女士们先生们，当我们踏上这段语言探索之旅时，请记住：2022年11月，ChatGPT不仅仅是登场亮相；它在语言领域的震撼程度堪比摇滚巨星的返场表演，为一场史诗般的文字盛宴拉开了序幕！如今，随着LLAMA-2、Falcon180B和Bard的出现，可以毫不夸张地说，大语言模型已成为数字时代的披头士乐队，让搜索引擎显得像是昨日旧闻。”

“但关键在于—如今各大公司都争先恐后地试图将这些大语言模型集成到自己的应用程序中。这就像一场淘金热，只不过我们不是在淘洗贵金属，而是在筛选那些闪耀着智慧光芒的语言字节。所以，朋友们，在这个文字掌握力量的时代，大语言模型就是那支强大的笔，我们不仅是在编写未来，更是在谱写一部注定畅销的语言传奇！欢迎来到大语言模型的时代，在这里我们正逐句改写规则，而文字就是变革的货币！”

一种更可靠的方式来强制规范提示词结构—在OpenAI和Anthropic的提示词工程指南中均有推荐（参见本节末尾的资源列表）—是使用XML风格的标签来标记提示词的不同部分。比如，之前的提示词可以写成下面这样：

<Persona>

您是一位经验丰富的大语言模型（LLM）开发者和知名演讲者。

</Persona>

<Context>

你受邀为一场大语言模型活动发表主题演讲。

</Context>

<Instruction>

为演讲撰写点睛之笔。

</Instruction>

<Input>

请包含以下事实：

- 大语言模型(LLM)自2022年11月ChatGPT发布以来已成为业界主流。
- 此后相继推出了LLaMA-2、Falcon180B、Bard等众多广受欢迎的大语言模型及基于LLM的聊天机器人。
- 大语言模型(LLM)的普及程度正逐步与搜索引擎比肩。
- 众多企业正积极将其自身应用与大语言模型进行集成。

</Input>

<Tone>

采用机智诙谐但引人入胜的语气。

</Tone>

<OutputFormat>

将文本分以每段五行,总共两段的格式呈现。

</OutputFormat>

注意 研究（例如，“提示词报告：提示词技术的系统性调查”，https://arxiv.org/abs/2406.06608）发现，明确命名提示词的不同部分往往能改善结果。然而，你并不需要为每个部分都命名。大多数大语言模型能够自行理解提示词中文本的意图。因此，你可以混合使用，为某些部分命名（例如，问题或示例），而省略其他部分的名称（例如，上下文或语气）。着手进行实验，看看哪种方式最适合你的使用场景和你所获得的结果。

如果你想更深入地研究提示词工程，我强烈推荐以下资源：

- https://mng.bz/z2aA
- https://mng.bz/0zVv
- https://mng.bz/KwmO
- https://mng.bz/9ynr
- www.promptingguide.ai/techniques
- https://github.com/dair-ai/Prompt-Engineering-Guide
- https://mng.bz/jZ8e

总结

- 提示词通过提供指令和上下文（背景信息，比如领域知识或约束条件）来引导大语言模型（LLM）的运行，清晰且具体的提示词往往能比模糊的提示词产生更优的结果。
- 不同指令模式适用于不同任务：分类型提示词要求给出类别标签或情感评分；生成型提示词要求创作原创性内容；抽取型提示词则从输入文本中提取特定事实。

- 当提示词未能产生预期输出时，大语言模型可能缺乏该类型任务的训练示例，此时添加少量示例或提升指令的清晰度，通常可有效解决问题。
- 少样本学习通过示例展示期望的响应模式：单样本（one-shot）使用一个示例，双样本（two-shot）使用两个示例，依此类推——增加示例数量通常能提升输出一致性，但也会相应增加token开销。
- 标准提示词通常包含以下六个要素：角色设定（角色）、上下文（背景）、指令（任务）、输入（数据）、风格（语气与格式）和示例（模式）。并非每个任务都需要全部包含这6个部分。
- 你可以根据需要省略、重新排列或扩展提示词的各个部分，并加入自定义元素，如约束条件、输出格式规范或多步推理指令。
- 基于API的提示词自动化通过将变量注入提示词模板，并以编程方式执行请求，从而将提示词应用从单次请求扩展至成千上万种变体。例如，只需使用一个模板，结合来自数据库或CSV文件的不同产品属性（如名称、类别、功能），即可批量生成数千条产品描述。
- 提示模板使用占位符（通常为用花括号包裹的`{variable_name}`），这些占位符在运行时会被替换为实际值，从而将提示此的结构与动态内容分离。
- LangChain 提供了三种提示词模板类，`PromptTemplate`用于基础的变量替换，支持将运行时动态数据注入预定义的模板结构中。`FewShotPromptTemplate`会根据输入内容的语义相似性，动态选择最相关的示例，实现少样本学习的自适应提示词构建。而`ChatPromptTemplate`专门用于结构化多轮对话场景。
- 思维链（CoT）提示词通过要求大语言模型在给出最终答案前，逐步展示其推理过程来提升输出的可解释性与准确性。通常可加入如“让我们逐步思考”之类的指令，或明确列出所需的推理步骤，引导模型进行结构化思考。
- 使用XML风格的标签来结构化提示词，明确划分各部分内容：`<persona>`、`<context>`、`<instruction>`、`<input>`、`<examples>`。

此方法遵循 OpenAI 与 Anthropic 的提示词工程规范，旨在提升提示词的清晰度与可操作性。

- 使用 OpenAI API 时，消息格式需以list列表形式组织，每条消息包含 role 和 content 两个字段：`messages=[{"role": "system", "content": "..."}, {"role": "user", "content": "..."}]` 支持的角色包括：system、user 和 assistant。

第二部分 摘要生成

本部分聚焦于将海量信息转化为清晰、简洁且实用的摘要成果。你将学习实用的文档摘要技术，适用于从大型报告到包含多种文件格式（如TXT、DOCX、PDF）的文件夹内容，同时严格遵守大语言模型的上下文长度限制。你将深入理解在何种场景下应选用 MapReduce 或精炼（Refine）策略，并掌握如何通过 LangChain 表达式语言（LCEL）将各组件无缝串联，构建出高效、准确且易于维护的自动化摘要流程。

从这里开始，你将超越简单的“归纳这段文本”式任务，迈向更高级的研究型应用场景。你将构建一个摘要引擎，它能自动搜索网络、收集相关信息，并基于提示词工程与模块化链，生成逻辑连贯、根基扎实的综合性报告。最终，你将借助LangGraph，将摘要工作流演进为智能体系统，引入显式状态管理与条件分支机制，使整个流水线具备智能适应能力、平滑扩展性，并为书中后续构建更自主的AI智能体奠定坚实基础。

3 使用LangChain进行文本摘要

本章涵盖以下内容

- 对超出大语言模型上下文窗口的大型文档进行摘要
- 跨多文档摘要

在第一章中，我们探讨了三种主要的LLM应用类型：摘要引擎、聊天机器人和AI智能体。在本章中，你将开始使用LangChain构建实用的摘要链，重点侧重于使用LangChain表达式语言（LCEL）来应对各种真实场景。链是一系列相互连接的操作，其中一步的输出成为下一步的输入一这对于自动化摘要等任务来说是理想的选择。这项工作为在下一章构建更高级的摘要引擎奠定了基础。

摘要引擎对于自动化处理大量文档的摘要任务至关重要，这项工作即使借助ChatGPT等工具进行人工处理，也既不切实际又成本高昂。从摘要引擎入手是开发大语言模型应用程序的一个实用切入点，它为更复杂的项目打下了坚实基础，并展现了LangChain的能力，我们将在后续章节中进一步探讨。

在我们开始构建之前，我们将探讨不同的摘要技术，每种技术都适用于特定场景，包括大型文档、内容整合和处理结构化数据。您已经在第2章中使用`PromptTemplate`处理过小型文档的摘要，因此我们将跳过这部分，专注于更复杂的示例。

3.1 为超出上下文窗口长度的文档生成摘要

如第一章所述，每个大型语言模型都有一个最大提示词尺寸，也称为*上下文窗口*。尽管流行的大语言模型的上下文窗口不断增长，您仍可

能遇到文档超出所选模型的token限制的情况。在这些情况下，您可以采用如图3.1所示的MapReduce方法。

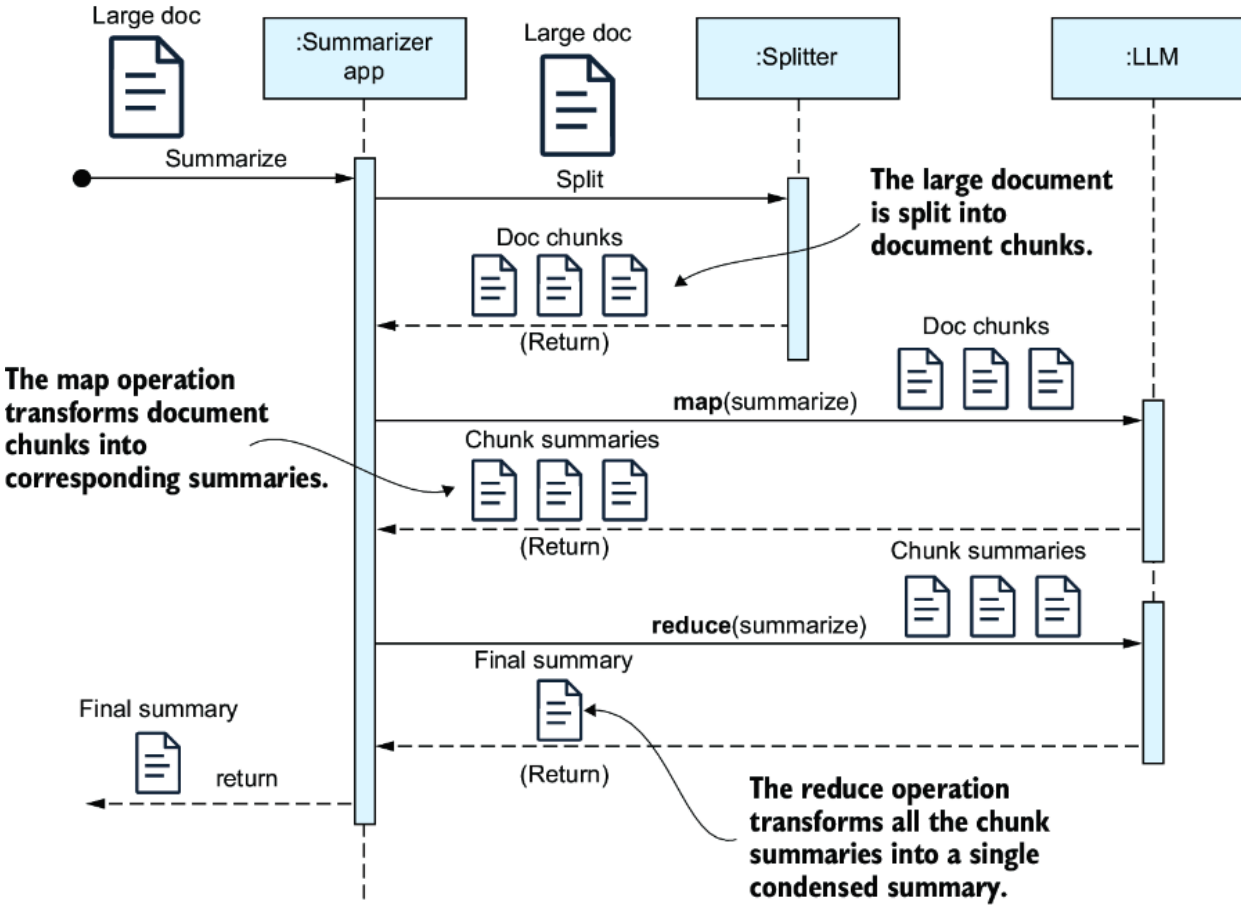

图 3.1 对超出大语言模型上下文窗口的文档进行摘要，需将其拆分为多个较小的文本块，分别对每个块进行摘要，再对这些块摘要的合并结果进行最终摘要。

> **定义** 大语言模型上下文窗口是指在单次提示词中可输入给语言模型的文本总量上限—包括指令和上下文。不同模型对上下文窗口支持的token数量各异，其中每个token大致对应一个单词。例如，GPT-3.5最多可处理16,000个token，GPT-4和Claude 3支持多达100,000个token，而更新的模型如GPT-5和Gemini可以处理超过100万个token。

当处理超出大语言模型上下文窗口的文档时，一种常见策略是将文本拆分为可管理的片段，为每个片段生成摘要，然后基于这些中间结果生成最终摘要。首先，你需要使用分词器将文本分割成片段。分词器

读取文本并将其拆分为token，这些token是文本的最小单位，通常是单词的一部分。对文档进行分词后，token会被分组为特定大小的片段。这让你可以控制大语言模型处理的内容大小，并确保token数量保持在你的大语言模型提示词限制内。在这里，我将向你展示如何使用`TokenTextSplitter`，它是`tiktoken`包的一部分，是由OpenAI开发的一个分词器。

首先打开终端，为本章节的代码创建一个名为`ch03`的新文件夹。然后，创建并激活一个虚拟环境：

```
C:\Github\building-llm-applications>md ch03
C:\Github\building-llm-applications>cd ch03
C:\Github\building-llm-applications\ch03>python -m venv env_c
h03
C:\Github\building-llm-applications\ch03>.\env_ch03\Scripts\a
ctivate
(env_ch03) C:\Github\building-llm-applications\ch03>
```

接下来，安装所需的包——`tiktoken`、`notebook`和`langchain`。如果您已从GitHub克隆了代码仓库（https://mng.bz/WwvW）或从曼宁出版社网站下载了代码压缩文件，请使用以下命令安装依赖：

```
C:\Github\building-llm-applications\ch03>pip install -r requi
rements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

安装完成后，启动Jupyter Notebook：

```
(env_ch03) C:\Github\Building-llm-applications\ch03>jupyter n
otebook
```

现在打开或创建一个Notebook，并将其命名为`03-summarization_examples.ipynb`。最后，保存该Notebook。

3.1.1 将文本切分为文档对象

让我们使用从古登堡计划网站（www.gutenberg.org）下载的文本文件`Moby-Dick.txt`来总结《白鲸记》这本书。你可以在我的GitHub页面本章节的子文件夹中找到`Moby-Dick.txt`文件（https://mng.bz/DwyV）。将此文件放到你的`ch03`文件夹中，并将文本加载到一个变量里：

```
with open("./Moby-Dick.txt", 'r', encoding='utf-8') as f:
    moby_dick_book = f.read()
```

> **注意**　请记住，在完整的《白鲸记》文本上运行代码可能会花费较高。提供的`Moby-Dick.txt`文件是一个简短版本，仅包含五个章节和大约18,000个token。使用此版本运行几次代码应该不会花费太多。然而，如果您打算进行多次测试，您可能希望进一步减小文件大小以节省费用。每次执行包含`LLMChain`模块的链时，您都会被计费。如果预算允许，您可以使用该书的完整版本，名称为`Moby-Dick_ORIGINAL_ EXPENSIVE.txt`。完整版的《白鲸记》文本大约有300,000个单词，或约350,000个token。使用GPT-5（每百万token费用为1.25美元），处理完整文本将花费约0.37美元。多次运行会累积费用。如果您切换到GPT-5-nano（每百万token费用为0.05美元），成本将降至约0.015美元，便宜得多了。

我将在第八章详细讨论分块策略一例如按大小和内容结构进行分块。现在，让我们将文本分割成每个约3000个token的块，以模拟比我们将要使用的GPT-5-nano模型更短的上下文窗口。

3.1.2 分割

开始分割之前，你需要做一些准备工作。首先，导入必要的库：

```
from langchain_openai import ChatOpenAI
from langchain_text_splitters import TokenTextSplitter
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda, Runnable
Parallel
import getpass
```

接下来，使用`getpass`函数获取您的OpenAI API密钥：

```
OPENAI_API_KEY = getpass.getpass('输入你的OPENAI_API_KEY')
```

系统将提示您输入您的`OPENAI_API_KEY`。之后，请在一个新的NoteBook单元格中实例化OpenAI模型：

```
llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,model_name="gp
t-5-nano")
```

现在，您已准备好设置第一条链，该链将文档分割成指定大小的块。在本章中，您将学习大语言模型表达式语言（LCEL）的基础知识，更详细的探讨将在下一章进行：

```
text_chunks_chain = (
    RunnableLambda(lambda x:
        [
            {
                'chunk': text_chunk,
            }
            for text_chunk in
               TokenTextSplitter(chunk_size=3000,
                                 ↪chunk_overlap=100).split_te
xt(x)
        ]
    )
)
```

注意　LangChain链是由实现了`Runnable`接口的组件构成的，这是一个Python抽象类，定义了组件如何接收输入、处理并返回输

出。任何实现了Runnable接口的类都可以成为链的一部分。RunnableLambda允许你将任何Python可调用对象转换为Runnable，从而轻松地将自定义函数集成到LangChain链中。它类似于Python的lambda表达式，你可以通过参数运行代码，并选择性地返回输出，而无需定义完整的函数。使用RunnableLambda，你可以创建一个链组件，而无需编写单独的类来实现Runnable接口。在本示例中，由RunnableLambda包装的代码通过x参数接收字符串形式的输入文本，并将其传递给split_text()函数，该函数将文本分割成块。

3.1.3 映射

下一步是设置映射链，它将为每个文档块运行一个摘要提示词，并确保在后续合并之前，文本的每一部分都能被独立处理：

```
summarize_chunk_prompt_template = """
编写以下文本的简洁摘要，并包含主要细节。
文本: {chunk}
"""

summarize_chunk_prompt =
↪PromptTemplate.from_template(summarize_chunk_prompt_templat
e)
summarize_chunk_chain = summarize_chunk_prompt | llm

summarize_map_chain = (
    RunnableParallel (
        {
            'summary': summarize_chunk_chain | StrOutputParse
r()
        }
    )
)
```

在这段代码中，管道操作符（|）用于将组件链接在一起，将一个对象的输出作为下一个对象的输入。例如，summarize_ chunk_prompt被管

道传输到大语言模型(llm)，这意味着生成的提示词被直接发送给模型。同样地，模型的输出被管道传输到StrOutputParser()，后者将模型的响应转换为干净的文本字符串。

> **注意** 在这个链中，我使用了 RunnableParallel，它类似于 RunnableLambda，但它作用于一个序列，并行处理每个元素。在本示例中，我们将文本块的序列输入到 summarize_map_chain，每个块将由内部的 summarize_ map_chain 并行进行摘要。

> ***MAPREDUCE***
>
> MapReduce是一种用于分两步处理大数据集的编程模型。首先，映射操作将数据分割成较小的子集，每个子集由相同的函数独立处理。此步骤通常返回按键分组的结果列表。接下来是合并操作，其中每个键的结果被聚合成单一结果。最终输出是一个键值对列表，其中键来自映射步骤，而值是在合并步骤中聚合映射数据返回的结果。

3.1.4 合并

设置合并链（用于汇总每个文档块摘要的过程）遵循与映射链类似的过程，但需要稍多的设置。首先定义提示词模板：

```
summarize_summaries_prompt_template = """
编写以下文本的简洁摘要，
该文本整合了多个摘要，并包含主要细节。
文本：{summaries}
"""

summarize_summaries_prompt =
↪PromptTemplate.from_template(summarize_summaries_prompt_temp
late)
```

接下来，您可以配置合并链：

```
summarize_reduce_chain = (
    RunnableLambda(lambda x:
        {
            'summaries': '\n'.join([i['summary'] for i in
x]),
        })
    | summarize_summaries_prompt
    | llm
    | StrOutputParser()
)
```

合并链包含一个lambda函数，它将映射链中的摘要合并为一个字符串。随后，该字符串由`summarize_ summaries_prompt`提示词处理，从而生成合并内容的最终摘要。

3.1.5 MapReduce组合链

最后，我们将文档分割链、映射链和合并链组合成一个单一的MapReduce链：

```
map_reduce_chain = (
   text_chunks_chain   #1
   | summarize_map_chain.map()   #2
   | summarize_reduce_chain  #3
)
```

#1 分割链
#2 映射链
#3 归约链

这种设置能高效地将输入文档分割成多个块，对每个块进行摘要，然后将这些摘要汇总成最终的摘要结果。`summarize_map_chain`上的`map()`函数对于实现块的并行处理至关重要。

3.1.6 MapReduce执行

一切现已准备就绪。使用以下命令启动大型文档的MapReduce摘要生成（如果您使用的是OpenAI免费套餐，此操作可能会因`429 Rate Limit`错误而失败）：

```
summary = map_reduce_chain.invoke(moby_dick_book)
```

如果你运行`print(summary)`，你将得到类似如下的输出：

```
《白鲸记》或《鲸鱼》电子书由古登堡计划发布，其引言概述了该书的获取方式和更新情况，首版电子书发布于2001年6月，最新版则在2021年8月。故事以叙述者伊希梅尔开篇，他为了摆脱忧郁状态而寻求海上慰藉，展现了海洋相较于城市生活的吸引力。他反思了自己加入捕鲸航行的原因，源于对鲸鱼的迷恋和对冒险的渴望。抵达新贝德福德后，伊希梅尔在寻找住宿时遇到困难，最终落脚于"喷水者客栈"，在那里他遭遇了混乱的环境和一位名为奎奎格的神秘鱼叉手。

当伊希梅尔与起初误认为自己是食人族的奎奎格同床共枕时，他逐渐克服了内心的恐惧。共度第一夜后的清晨，凸显出他们之间奇特却悄然滋长的羁绊，伊希梅尔观察到奎奎格独特的习俗和彬彬有礼的举止，由此深刻揭示了命运、选择，以及捕鲸业中对未知世界的迷人吸引力
```

> **警告** 如前所述，运行`map_reduce_chain`会产生费用；您想要生成摘要的文本越长，费用就越高。因此，如果您想控制开支，可以考虑进一步缩短输入文本（在我们的例子中，是使用`Moby-Dick.txt`电子书文件）。此外，请确保您的OpenAI API信用余额保持为正数，以避免出现如下错误：`RateLimitError: Error code: 429 - {'error': {'message': 'You exceeded your current quota, please check your plan and billing details [...]。'`如有必要，请登录OpenAI API页面，导航到设置 > 账单，并设置至少5美元的信用额度。

现在让我们进入下一个使用场景：跨文档摘要。在这个使用场景中，目标是从多个来源整合信息，而不是处理单个长文本。

3.2 跨文档摘要

您可以轻松学习如何汇总来自不同数据源的信息，例如维基百科或本地文件（包括Microsoft Word、PDF和文本格式）。如图3.2所示，此过程类似于上一节中使用的MapReduce技术。

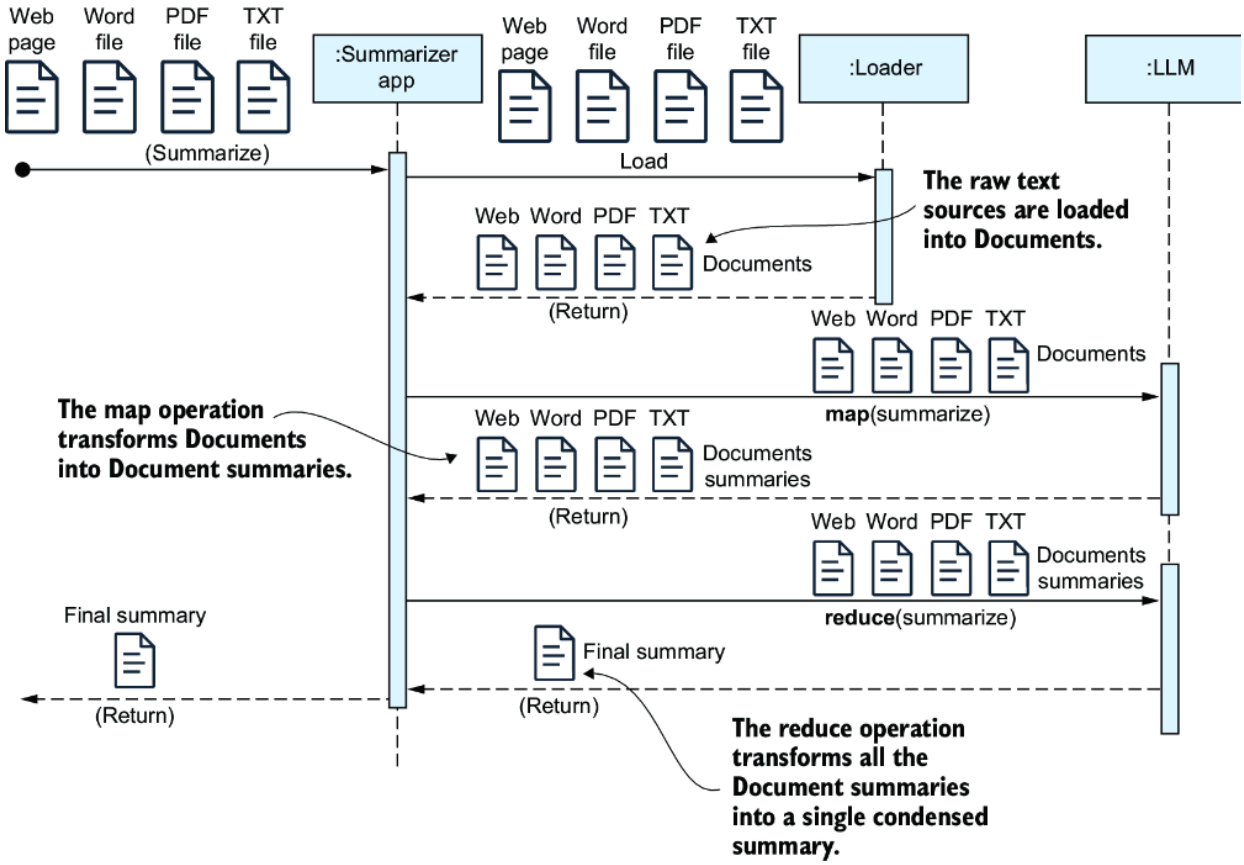

图 3.2 使用先前介绍的MapReduce技术对多篇文档生成摘要。在此方法中，每个文档片段会经历一个`map`操作以生成摘要。然后，这些独立的摘要通过`reduce`操作进一步合并为一份单一的摘要。

在图3.2的时序图中，来自每个原始文本源的内容被加载到相应的`Document`实例中。在`map`操作期间，这些`Document`对象被转换为独立的摘要，然后在`reduce`操作期间合并成一个单一的摘要。

接下来，我将向您介绍另一种*精炼*技术，如图3.3所示。采用这种方法，最终摘要通过迭代方式对当前最终摘要与其中一个文档块的组合

进行归纳来逐步构建。这个过程持续进行，直到所有文档块都被处理完毕，从而完成最终摘要。

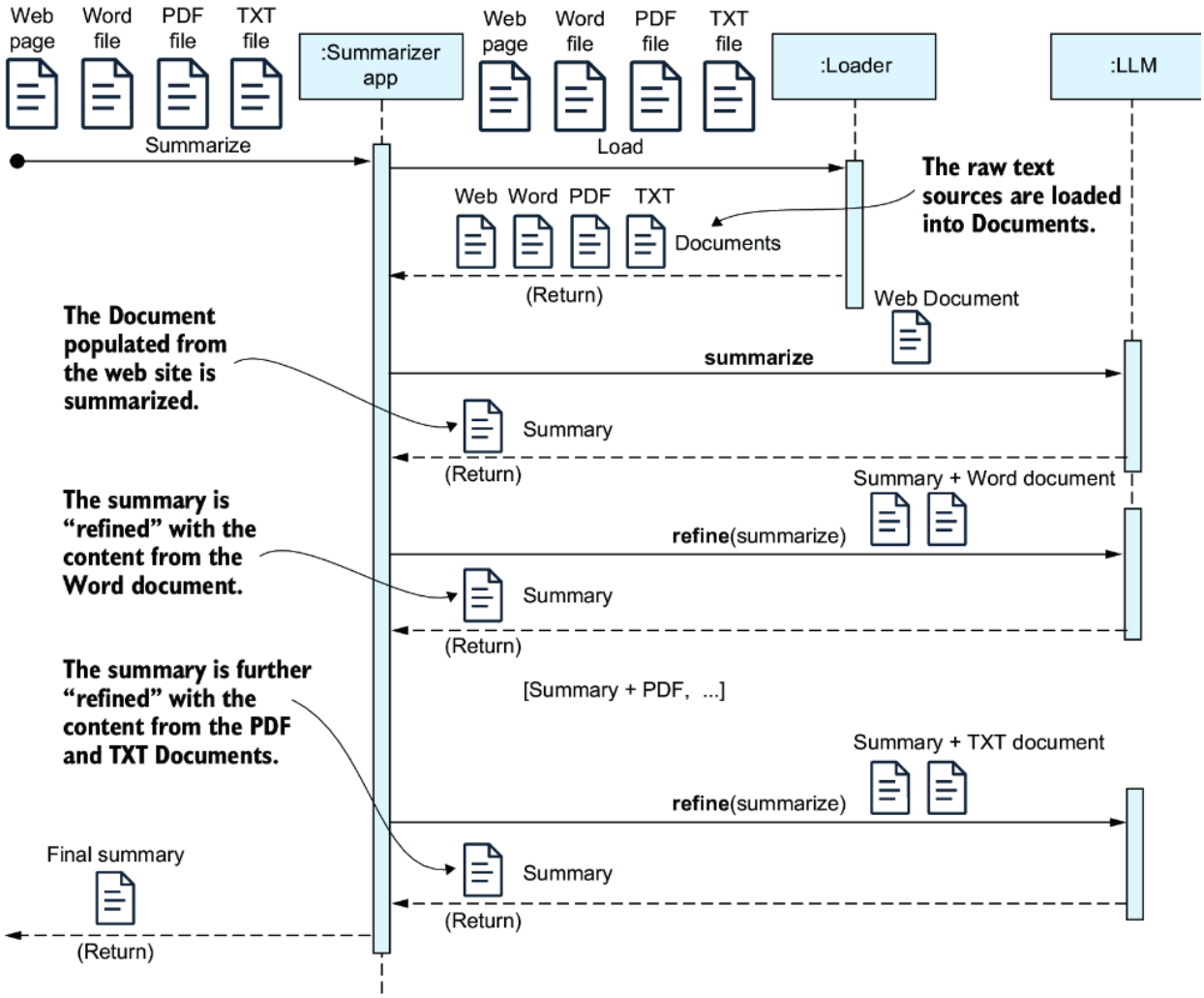

图 3.3 使用精炼技术跨文档生成文本摘要

在这种方法中，您通过逐步完善来构建最终摘要。每份文档都会连同当前摘要草稿一起发送给大语言模型进行归纳。这一过程持续进行，直到所有文档处理完毕，从而得出最终摘要。MapReduce 方法非常适合归纳大量文本，在处理负荷可接受的情况下，即使损失部分内容也无妨。相比之下，当您希望确保最终摘要能完整捕捉每个部分的精髓时，精炼技术则更为适用。

3.2.1 创建文档对象列表

在归纳一份大型文档时，通常需要先将其拆分为较小的片段，每个片段视为独立的文档。在本示例中，我们已有一组现成的文档，因此无需进行任何拆分。如何创建每个`Document`对象将取决于文本的来源。我将向您展示如何归纳来自四种不同来源的内容：一个维基百科页面以及存储在本地文件夹中的多种格式文件（TXT、DOCX、PDF）。所有内容都与Paestum相关，这是公元前500年左右位于意大利南部奇伦托海岸的一个希腊殖民地。您将为每个数据源使用相应的`DocumentLoader`，这些加载器选自第1章第1.3节中介绍的众多LangChain`Document`对象模型。

3.2.2 维基百科内容

让我们从维基百科的内容开始。虽然你可以使用`WebBaseLoader`从基于网络的数据内容创建文档，但特定的加载器是为从特定网站检索内容而定制的，例如用于互联网电影剧本数据库（IMSDb）网站的`IMSDbLoader`，用于AZLyrics网站的`AZLyricsLoader`，以及用于维基百科网站的`WikipediaLoader`。

如果您遵循了3.1节开头的软件包安装说明，那么您应该已经安装了必要的加载器包，包括用于Word文件的`docx2txt`（由`Docx2txtLoader`使用）和用于PDF的`pypdf`（由`PyPDFLoader`使用）。现在，您可以从Paestum维基百科页面导入内容：

```
from langchain.document_loaders import WikipediaLoader

wikipedia_loader = WikipediaLoader(query="Paestum", load_max_
docs=2)
wikipedia_docs = wikipedia_loader.load()
```

> **注意** `WikipediaLoader`可能会从请求文章中引用的其他维基百科超链接加载内容。例如，Paestum文章引用了Paestum国家考古博物馆、卢卡尼亚地区、卢卡尼亚人以及赫拉和雅典娜神庙，从

而导致加载了额外内容。因此，它返回的是一个Document列表，而非单个Document对象。我已将返回文档的最大数量设置为2以节省摘要生成成本，但您可以根据需要进行调整。

3.2.3 基于文件的内容

首先，从GitHub下载或拉取Paestum文件夹，并将其放置到您本地的ch03目录中（如果您没有克隆整个代码仓库）。ch03目录内的Paestum子文件夹包含三个文件：

- Paestum-Britannica.docx—内容源自大英百科全书网站。
- PaestumRevisited.pdf—节选自斯德哥尔摩大学提交的硕士论文《重访Paestum》。该节选仅包含四页，但您可以使用位于同一文件夹中的完整文档。（PaestumRevisited-StocholmsUniversitet.pdf）.
- Paestum-Encyclopedia.txt—内容来自于网站Encyclopedia.com.

以下是加载这些文件到对应文档的流程：

```
from langchain.document_loaders import Docx2txtLoader
from langchain.document_loaders import PyPDFLoader
from langchain.document_loaders import TextLoader

word_loader = Docx2txtLoader("Paestum/Paestum-Britannica.doc
x")
word_docs = word_loader.load()

pdf_loader = PyPDFLoader("Paestum/PaestumRevisited.pdf")
pdf_docs = pdf_loader.load()

txt_loader = TextLoader("Paestum/Paestum-Encyclopedia.txt")
txt_docs = txt_loader.load()
```

文档变量（word_docs、pdf_docs、txt_docs）之所以采用复数形式，是因为加载器总是返回一个文档列表，即使该列表仅包含一个元素。

注意　您可能已经注意到，我们使用`TextLoader`直接从`Paestum-Encyclopedia.txt`创建了一个`Document`对象数组，并且你可能会疑惑为什么之前`Moby-Dick.txt`文件是用Pythonfile阅读器读取的。在那种情况下，目的是将内容分割成特定数量的token以适应大语言模型提示词，因此需要手动为每个部分创建`Document`对象。

3.2.4 创建文档列表

现在，您可以将来自不同来源的所有文档合并到一个单一的`Document`列表中。使用如下代码来实现：

```
all_docs = wikipedia_docs + word_docs + pdf_docs + txt_docs
```

当所有内容都编译完成后，您就可以使用精炼技术来归纳内容了。下一步是创建一个生成最终摘要的链。

文档加载器

除了针对特定数据源的文档加载器，我也建议您探索一下`UnstructuredLoader`。它能让您从多种文件类型中导入内容，包括Word、PDF和TXT文件等。

另一个选项是`DirectoryLoader`，它在内部使用了`UnstructuredLoader`。它允许您通过单次操作从同一文件夹中不同格式的文件加载内容。

作为一项练习，我建议使用`UnstructuredLoader`或`DirectoryLoader`重新创建来自Word、PDF和TXT格式的Paestum内容文档。如果你选择这样做，你需要安装相关包并参考LangChain网站上的文档：

```
pip install "unstructured[all-docs]"
```

LangChain框架提供了多种加载器，用于从各类数据源中检索内容。我强烈建议您浏览该列表，并尝试任何您感兴趣的加载器：https://mng.bz/EwmR。

3.2.5 逐步完善最终摘要

现在一切准备就绪，您可以创建一个链来逐步生成最终摘要。首先导入必要的模块：

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
import getpass
```

接下来，获取 `OPENAI_API_KEY`，并像之前一样设置大语言模型：

```
OPENAI_API_KEY = getpass.getpass('输入你的OPENAI_API_KEY')

llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY,model_name="gp
t-5-nano")
```

现在定义用于归纳单个文档的相关提示链：

```
doc_summary_template = """为以下文本撰写一份简洁的摘要：
{text}
文档摘要："""
doc_summary_prompt = PromptTemplate.from_template(doc_summary
_template)

doc_summary_chain = doc_summary_prompt | llm
```

接下来，通过迭代方式将当前摘要与额外文档的摘要相结合，来设置用于优化摘要的链：

```
refine_summary_template = """
你必须根据当前已生成的精炼摘要以及一份额外文档的内容，生成最终摘要。
这是当前已生成的精炼摘要：
{current_refined_summary}
这是额外文档的内容：{text}
仅当额外文档的内容有用时才使用它，否则直接返回当前完整的摘要。"""

refine_summary_prompt =
↪PromptTemplate.from_template(refine_summary_template)

refine_chain = refine_summary_prompt | llm | StrOutputParser
()
```

最后，定义一个函数，循环遍历每个文档，使用`doc_summary_chain`对其进行摘要，并使用`refine_chain`来精炼整体摘要：

```
def refine_summary(docs):

    intermediate_steps = []
    current_refined_summary = ''
    for doc in docs:
        intermediate_step = \
           {"current_refined_summary": current_refined_summar
y,
            "text": doc.page_content}
        intermediate_steps.append(intermediate_step)

        current_refined_summary = refine_chain.invoke(interme
diate_step)

    return {"final_summary": current_refined_summary,
            "intermediate_steps": intermediate_steps}
```

现在，您可以通过在准备好的文档列表上调用`refine_summary()`来启动摘要生成过程：

```
full_summary = refine_summary(all_docs)
```

打印 `full_summary` 对象将显示 `final_summary` 下的最终摘要以及 `intermediate_steps` 下的中间步骤。虽然为了方便起见，结果步骤被缩短了，但我鼓励您观察摘要是如何在每个阶段演变的：

```
print(full_summary )
```

以下是输出内容的一个摘录：

```
{'final_summary': "**最终摘要:**\n\nPaestum是一座古希腊城市，位于大希腊地区的第勒尼安海沿岸，约公元前600年由锡巴里斯的定居者建立，最初命名为波塞冬尼亚。这座城市作为希腊定居点而繁荣发展[…已缩写…]那些对古希腊文化和建筑感兴趣的人。", 'intermediate_steps': [{'current_ refined_summary': '', 'text': 'Paestum (PEST-əm，美式英语也读作PEE-stəm) 是大希腊地区第勒尼安海沿岸的一座重要古希腊城市。Paestum的废墟因其三座大约建于公元前550年至450年的多立克式古希腊神庙而闻名，这些神庙保存状况极佳。城墙和圆形剧场[…已缩写…]
```

我们已经介绍了几种摘要技术，现在让我们稍作停顿，回顾一下所学内容。这是一个很好的时机，可以退一步思考这些方法彼此之间有何关联，以及每种方法在何时使用最为有效。

3.3 摘要流程图

为了归纳本章内容，我提供了一个流程图来帮助你根据具体需求选择最合适的摘要技术，如图3.4所示。这里，第一个关键决策是你要归纳的是一个文档还是多个文档。如果只是一个文档且其内容在上下文窗口容量内，你可以将整个文档"塞入"单个提示词中进行摘要。如果容量不够，则使用MapReduce方法。对于多个文档，如果它们都能放入上下文窗口，你也可以将它们塞入单个提示词中。如果不行，对于大量文档可以使用MapReduce方法，或者如果你想确保每个文档的核心内容都包含在最终摘要中，则使用精炼技术。

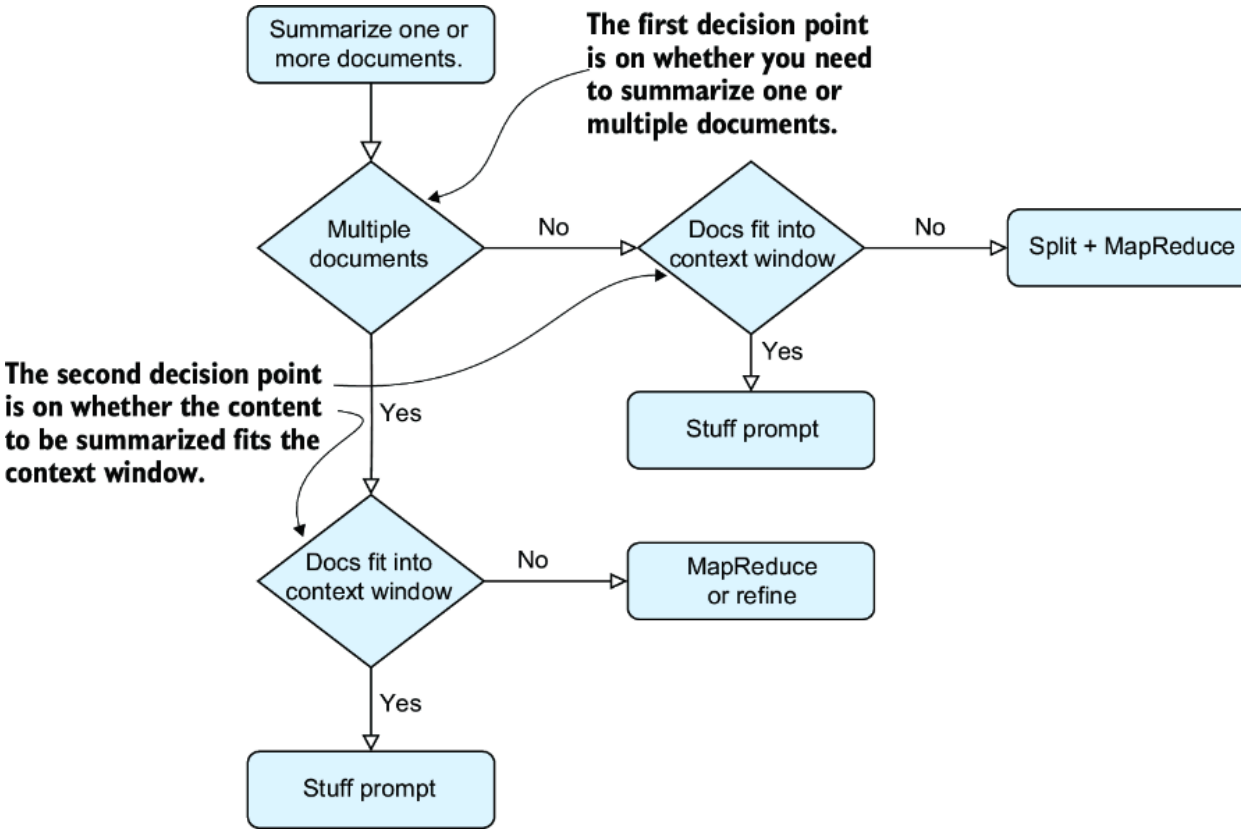

图 3.4 摘要生成流程图。此流程图指导您根据是否需要总结多个不相关的文档以及输入文本是否适配上下文窗口，来选择正确的摘要生成方法。

总结

- 文本摘要通过压缩文档内容，在保留关键信息的前提下生成更简短的版本，广泛应用于执行报告、文章摘要和内容预览等场景。具体策略需根据文档的数量与规模灵活选择：对于单篇短文本，可直接采用提示词驱动的方式；中等长度文本适合使用链式处理；而当面对超出上下文窗口限制的大型语料库时，则需采用 MapReduce 或精炼（Refine）策略进行高效摘要。
- LangChain的`Document`对象将原始文本与元数据（如来源、页码、时间戳）封装在一起，以在处理流程中完整保留内容的溯源信息。
- MapReduce 摘要方法在映射（map）阶段独立且并行地处理各文本块，在合并（reduce）阶段将部分摘要合并为最终输出。该策略适用于处理超出大语言模型上下文窗口限制的大型文档，例如百页以上的报告。当任务优先考虑并行处理的速度，而非完全保留各文本块间的上下文关联时，应优先选用 MapReduce 方法。
- 精炼摘要技术通过依次纳入每个新文档片段，迭代更新当前摘要；每一步都同时考虑当前摘要与下一个文本块。
- MapReduce 摘要方法为换取并行处理速度与更低的token开销，会牺牲摘要的完整性；而精炼（Refine）技术虽能更好地保留上下文信息，但因其串行处理特性，会导致延迟增加和总token消耗上升。
- 使用`PromptTemplate.from_template(template_ string)`创建提示词模板，并通过管道操作符串联组件—例如 `summarize_chunk_chain = summarize_chunk_prompt` | llm—可将提示词的输出直接传递给大语言模型。
- 为构建用于并行处理的映射(Map)链，需先定义一个处理每个文本块的`summarize_map_chain`，再通过 `RunnableParallel` 的`.map()`方法，将该链同时应用于所有文本块。